

CONECTAFP

HECHO POR: IKER MAGRO JUÁREZ

CURSO ACADÉMICO: 2025/2026

CONVOCATORIA: ORDINARIA

FECHA DE ENTREGA:

TUTOR/A: CONSUELO MOYA GUTIERREZ

MÓDULO PROFESIONAL: DESARROLLO DE APLICACIONES WEB (DAW)

RESUMEN

ConectaFP es una aplicación web diseñada para conectar alumnos de Formación Profesional en España con empresas que ofrecen prácticas. La plataforma tiene tres tipos de usuario: alumnos, profesores y empresas.

La empresa puede publicar ofertas de prácticas mediante la especificación de la modalidad (presencial, remoto o híbrido) y los requisitos. Los alumnos pueden explorar y buscar ofertas y enviar solicitudes. Cuando una solicitud es aceptada, se genera una tutoría que vincula al alumno con la empresa y con un profesor del centro educativo, quien va a supervisar el seguimiento. Al finalizar la tutoría, se registra una valoración.

Este proyecto surge como respuesta a una problemática común: los alumnos tienen dificultades a la hora de encontrar prácticas en una empresa. Por lo tanto, la solución propuesta es crear una aplicación web o red social para facilitar tanto a la empresa como al alumno asignar prácticas. El público objetivo son alumnos que desean encontrar prácticas de manera sencilla y rápida.

Para el desarrollo se ha seguido una metodología basada en el ciclo de vida del software, incluyendo análisis de requisitos, diseño, codificación, pruebas y validación. Se han utilizado tecnologías como MySQL, HTML, CSS y frameworks como React y Laravel.

La elaboración del proyecto ha requerido aproximadamente tres meses. Como conclusión, se destaca la utilidad del sistema para el usuario final y el potencial de ampliación a plataformas móviles.

ABSTRACT

ConectaFP is a web application designed to connect Vocational Training students in Spain with companies offering internships. The platform has three types of users: students, teachers, and companies.

Companies can post internship opportunities by specifying the format (in-person, remote, or hybrid) and requirements. Students can browse and search for opportunities and submit applications. When an application is accepted, a mentoring session is generated, linking the student with the company and a teacher from the educational institution, who will monitor the student's progress. At the end of the mentoring session, an evaluation is recorded.

This project arose in response to a common problem: students have difficulty finding internships. Therefore, the proposed solution is to create a web application or social network to facilitate the process for both companies and students to assign internships. The target audience is students who want to find internships easily and quickly.

The development followed a methodology based on the software development life cycle, including requirements analysis, design, coding, testing, and validation. Technologies such as MySQL, HTML, CSS, and frameworks like React and Laravel were used.

The project took approximately three months to develop. In conclusion, the system's usefulness to the end user and its potential for expansion to mobile platforms are highlighted.

ÍNDICE

1. JUSTIFICACIÓN.....	5
2. OBJETIVOS.....	5
3. METODOLOGÍAS.....	5
3.1 ANÁLISIS DE REQUISITOS.....	5
3.2 DISEÑO DEL SISTEMA.....	6
3.3 DISEÑO DEL PROGRAMA.....	12
3.4 CODIFICACIÓN.....	34
3.5 EJECUCIÓN DE PRUEBAS.....	37
3.6 VERIFICACIÓN.....	38
3.7 MANTENIMIENTO.....	38
4. TECNOLOGÍAS.....	39
5. PLANIFICACIÓN DEL PROYECTO.....	39
6. DESARROLLO Y EJECUCIÓN DEL PROYECTO.....	40
7. CONCLUSIONES.....	41
8. LÍNEAS DE INVESTIGACIÓN Y DESARROLLO FUTURAS.....	41
9. BIBLIOGRAFÍA.....	41
10. EXTRAS.....	42
10.1 ANÁLISIS DE MERCADO.....	42
10.2 SEGURIDAD Y PROTECCIÓN DE DATOS.....	42
10.3 DESPLIEGUE.....	45
11. ANEXO - MANUAL DE USUARIO.....	52

1. JUSTIFICACIÓN

El proyecto surge debido a que los alumnos tienen dificultades a la hora de buscar y explorar empresas que permitan realizar prácticas, lo cual representa una barrera. Por ello, ConectaFP propone una solución: implementar una página web o red social para que el alumno explore o busque empresas que permitan solicitar y realizar prácticas de manera sencilla y rápida.

2. OBJETIVOS

Objetivo principal: implementar una aplicación web que permita al alumno de Formación Profesional explorar y solicitar prácticas en empresas de manera sencilla y rápida.

Objetivos específicos:

1. Crear un sistema para que las empresas publiquen sus ofertas de prácticas.
2. Permitir a los profesores solicitar prácticas para sus alumnos a las empresas.
3. Proporcionar herramientas para que gestionen los profesores las tutorías con sus alumnos.

3. METODOLOGÍAS

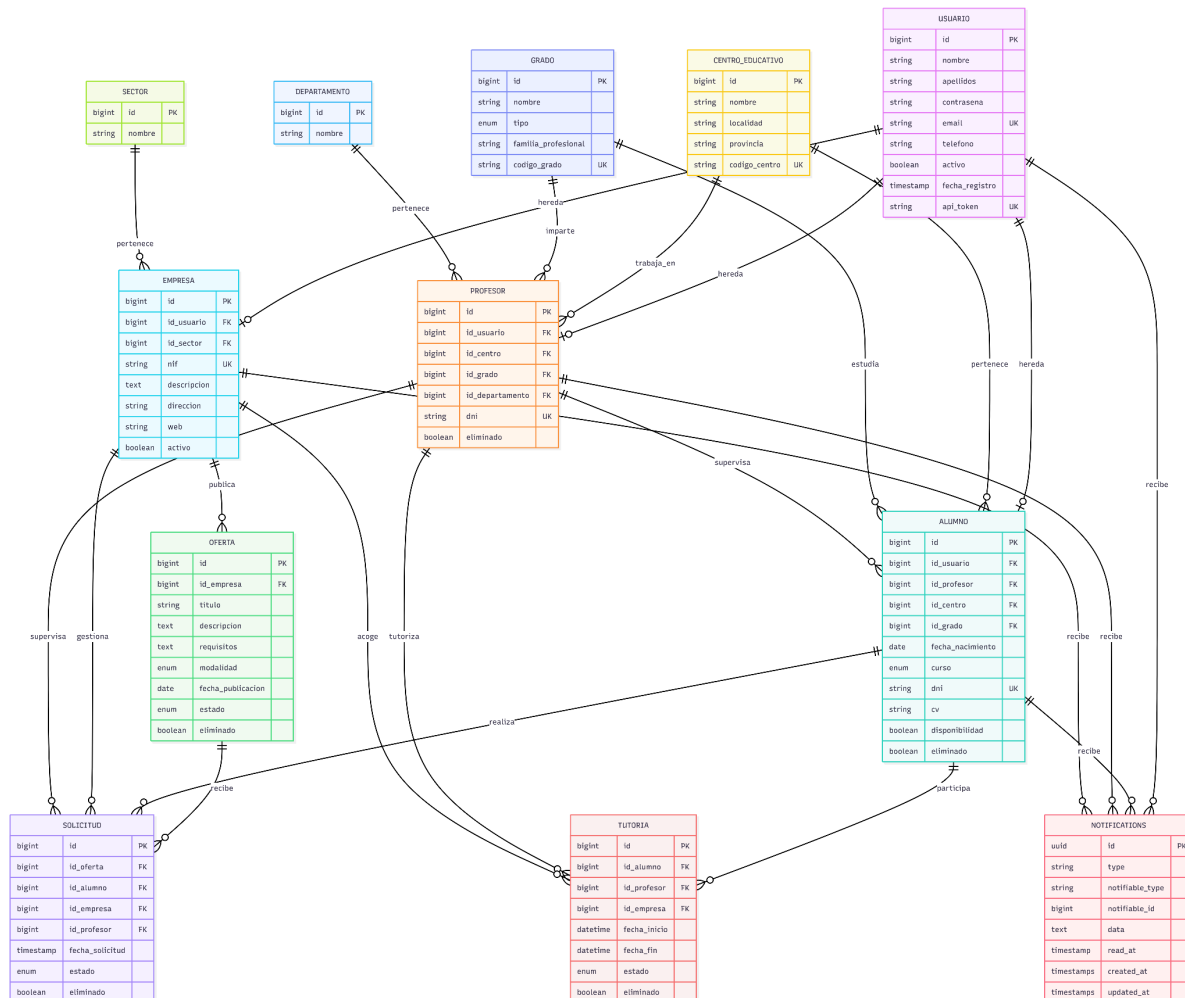
3.1 ANÁLISIS DE REQUISITOS

- Requisitos funcionales:
 - El sistema debe permitir el registro de usuarios mediante distintos roles (Alumno, Profesor y Empresa).
 - Los alumnos deben poder consultar ofertas de prácticas disponibles.
 - Las empresas deben poder publicar ofertas de prácticas.
 - El sistema debe gestionar solicitudes de prácticas.
 - Los profesores deben supervisar a los alumnos asignados.
 - Debe existir un sistema de notificaciones para mantener informados a los usuarios.
- Requisitos no funcionales:
 - Interfaz intuitiva.
 - Persistencia de datos segura.

3.2 DISEÑO DEL SISTEMA

Base de datos

DIAGRAMA DEL MODELO ENTIDAD-RELACIÓN



Para poder empezar a explicar el modelo entidad-relación necesitamos saber y conocer las distintas entidades:

- **Usuario:** aquella entidad que va a representar la base para todos los roles de usuario del sistema. Sus atributos son:
 - **id:** identificador de clave primaria (PK).
 - **nombre:** nombre del usuario.
 - **apellidos:** apellidos del usuario.
 - **contrasena:** contraseña encriptada.
 - **email:** email único para cada usuario (UNIQUE).
 - **telefono:** número de teléfono.
 - **activo:** estado del usuario (activo/inactivo).
 - **fecha_registro:** timestamp automático de registro.

- api_token: token único para cada usuario para la autenticación del usuario mediante API.
- Alumno: aquella entidad que va a representar un usuario con el rol de alumno o estudiante de Formación Profesional. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - id_usuario: identificador de clave foránea que hace referencia al usuario (FK).
 - id_profesor: identificador de clave foránea que hace referencia al profesor asignado (FK).
 - id_centro: identificador de clave foránea que hace referencia al centro educativo donde estudia el alumno (FK).
 - id_grado: identificador de clave foránea que hace referencia al grado formativo que está cursando el alumno (FK).
 - fecha_nacimiento: fecha de nacimiento del alumno.
 - curso: curso que está realizando (1º/2º).
 - dni: documento de DNI del alumno.
 - disponibilidad: si el alumno está disponible para realizar prácticas.
 - eliminado: borrado lógico.
- Profesor: aquella entidad que va a representar un usuario con el rol de profesor, es decir, el docente que va a supervisar las prácticas de los alumnos asignados:
 - id: identificador de clave primaria (PK).
 - id_usuario: identificador de clave foránea que hace referencia al usuario (FK).
 - id_centro: identificador de clave foránea que hace referencia al centro educativo donde trabaja (FK).
 - id_grado: identificador de clave foránea que hace referencia al grado formativo que imparte (FK).
 - id_departamento: identificador de clave foránea que hace referencia al departamento al que pertenece.
 - dni: documento de DNI del profesor.
 - eliminado: borrado lógico.
- Empresa: aquella entidad que va a representar un usuario con el rol de empresa, es decir, aquellas entidades que van a ofrecer las prácticas. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - id_usuario: identificador de clave foránea que hace referencia al usuario (FK).
 - id_sector: identificador de clave foránea hace referencia al sector empresarial al que pertenece la empresa (FK).
 - nif: documento del identificador fiscal único de la empresa.
 - descripcion: descripción de la empresa.
 - direccion: dirección de la sede física de la empresa.

- web: página web de la empresa.
- activo: estado de la empresa (activo/inactivo).
- Oferta: aquella entidad que va a representar las ofertas de prácticas publicadas por las empresas. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - id_empresa: identificador de clave foránea que hace referencia a la empresa que publica las ofertas (FK).
 - titulo: titulo de la oferta.
 - descripcion: descripción detallada de la oferta.
 - requisitos: requisitos para cumplir y aplicar la oferta.
 - modalidad: modalidad de trabajo (presencial, remoto o híbrido).
 - fecha_publicacion: fecha de publicación de la oferta.
 - estado: estado de la oferta (abierta, cerrada, pausada).
 - eliminado: borrado lógico.
- Solicitud: aquella entidad que va a representar todas las peticiones de los profesores a las empresas para que los alumnos asignados realicen las prácticas. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - id_oferta: identificador de clave foránea que hace referencia a la oferta solicitada (FK).
 - id_alumno: identificador de clave foránea que hace referencia al alumno asignado por el profesor que ha solicitado la oferta (FK).
 - id_empresa: identificador de clave foránea que hace referencia a la empresa que ha publicado la oferta (FK).
 - id_profesor: identificador de clave foránea que hace referencia al profesor que supervisa (FK).
 - fecha_solicitud: timestamp automático de la solicitud de la oferta.
 - estado: estado de la solicitud (revisión, aceptada o rechazada).
 - eliminado: borrado lógico.
- Tutoría: aquella entidad que va a representar la supervisión de los profesores hacia los alumnos que están realizando las prácticas en las empresas. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - id_alumno: identificador de clave foránea que hace referencia a los alumnos que están en prácticas (FK).
 - id_profesor: identificador de clave foránea que hace referencia a los profesores o tutores que tienen asignados a los alumnos en las prácticas (FK).
 - id_empresa: identificador de clave foránea que hace referencia a las empresas que acogen los alumnos asignados por los profesores (FK).

- fecha_inicio: inicio del evento de la tutoría.
 - fecha_fin: fin del evento de la tutoría.
 - estado: estado de la tutoría (activa, finalizada, cancelada).
 - eliminado: borrado lógico.
- Centro educativo: aquella entidad auxiliar que va a representar todos los centros educativos de Formación Profesional disponibles en España. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - nombre: nombre del centro educativo.
 - localidad: ciudad o localidad donde se encuentre el centro educativo.
 - provincia: provincia a la que pertenezca el centro educativo.
 - codigo_centro: código único del centro (UNIQUE).
- Grado: aquella entidad auxiliar que va a representar todos los grados formativos disponibles de Formación Profesional en España. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - nombre: nombre del grado formativo.
 - tipo: tipo de grado formativo a que pertenezca (grado medio o grado superior).
 - familia_profesional: familia a la que pertenece.
 - codigo_grado: código oficial único (UNIQUE).
- Sector: aquella entidad auxiliar que va a representar todos los sectores empresariales. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - nombre: nombre del sector empresarial.
- Departamento: aquella entidad auxiliar que va a representar todos los departamentos de los centros educativos. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - nombre: nombre del departamento.
- Notifications: aquella entidad que va a representar el sistema de notificaciones que ofrece Laravel. Sus atributos son:
 - id: identificador de clave primaria (PK).
 - type: tipo de notificación.
 - data: datos de la notificación.
 - read_at: timestamp de lectura, es decir, la fecha cuando el usuario ha leído la notificación.
 - created_at y updated_at: timestamps automáticos.

Ya comprendido las entidades, se necesita darse las siguientes relaciones:

- Usuario -> Alumno (1:1): un usuario puede ser un alumno.
- Usuario -> Profesor (1:1): un usuario puede ser un profesor.
- Usuario -> Empresa (1:1): un usuario puede ser una empresa.
- Centro educativo -> alumno (1:N): un centro educativo puede tener muchos alumnos, pero un alumno únicamente puede estar en un centro educativo.
- Centro educativo -> Profesor (1:N): un centro educativo puede tener muchos profesores, pero un profesor únicamente puede estar en un centro educativo.
- Grado -> Alumno (1:N): un grado formativo lo pueden cursar muchos alumnos.
- Grado -> Profesor (1:N): un grado formativo puede ser impartido por varios profesores.
- Departamento -> Profesor (1:N): un departamento tiene varios profesores.
- Sector -> Empresa (1:N): un sector empresarial puede agrupar varias empresas.
- Profesor -> Alumno (1:N): un profesor puede supervisar a varios alumnos.
- Empresa -> Oferta (1:N): una empresa puede publicar varias ofertas.
- Oferta -> Solicitud (1:N): una oferta puede recibir varias solicitudes.
- Alumno -> Solicitud (1:N): un alumno puede hacer varias solicitudes.
- Empresa -> Solicitud (1:N): una empresa puede gestionar varias solicitudes.
- Profesor -> Solicitud (1:N): un profesor puede supervisar varias solicitudes.
- Alumno -> Tutoría (1:N): un alumno puede tener varias tutorías.
- Profesor -> Tutoría (1:N): un profesor puede tutorizar a varios alumnos.
- Empresa -> Tutoría (1:N): una empresa puede acoger varias tutorías.
- Notifications: puede notificar a cualquier usuario (alumno, profesor, empresa).

Se ha utilizado las migraciones ofrecidas por Laravel para almacenar la base de datos. En este caso, fue útil para guardar la base de datos con el motor de base de datos MySQL. Ha proporcionado herramientas para migrar los datos a la base de datos sin tener que depender de realizar las tablas y relaciones con código SQL, debido a que Laravel ofrece herramientas para crearlas de manera sencilla y rápida.

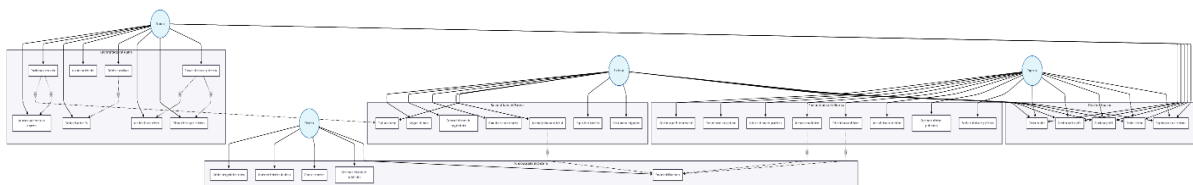
```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration {
8      /**
9       * Run the migrations.
10      */
11      public function up(): void
12      {
13          Schema::create('usuario', function (Blueprint $table) {
14              $table->id();
15              $table->string('nombre', 50)->nullable(false);
16              $table->string('apellidos', 100)->nullable();
17              $table->string('contrasena', 255)->nullable(false);
18              $table->string('email', 100)->nullable(false)->unique();
19              $table->string('telefono', 20)->nullable(false);
20              $table->boolean('activo')->default(1);
21              $table->timestamp('fecha_registro')->useCurrent();
22              $table->string('api_token', 80)->unique()->nullable();
23          });
24      }
25
26      /**
27       * Reverse the migrations.
28      */
29      public function down(): void
30      {
31          Schema::dropIfExists('usuario');
32      }
33  };

```

Imagen representativa de un ejemplo de migración a base de datos de la tabla usuario mediante Laravel

Casos de uso



Para poder explicar el diagrama de casos de uso de este sistema, se ha desglosado de la siguiente manera:

- Actores del sistema
 - Alumno: estudiantes de Formación Profesional que buscan realizar prácticas en empresas cuyos objetivos son encontrar y solicitar prácticas.
 - Profesor: docentes que supervisan y tutorizan a los alumnos durante las prácticas cuyo objetivo es supervisar a los alumnos asignados en las prácticas.
 - Empresa: entidades que ofrecen puestos de prácticas para estudiantes de Formación Profesional cuyos objetivos son publicar ofertas y seleccionar candidatos.

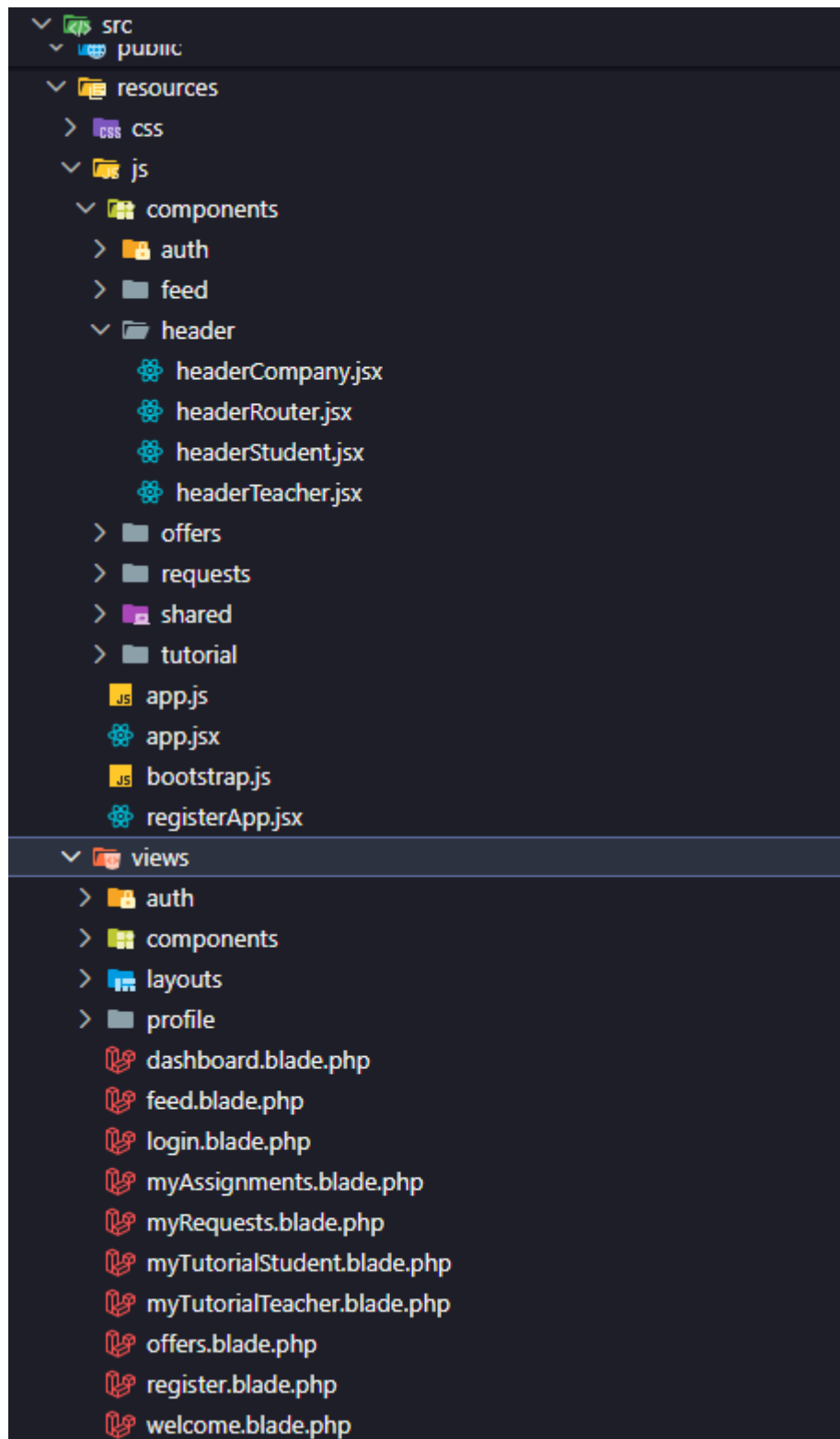
- Gestión de usuarios (todos los actores)
 - Registrarse en el sistema: seleccionar tipo de usuario, completar formulario de registro y acceder al sistema.
 - Iniciar sesión en el sistema.
 - Cerrar sesión.
- Funcionalidades del alumno
 - Buscar ofertas de prácticas: explorar ofertas disponibles en la plataforma.
 - Filtrar ofertas: aplicar filtros por sector, modalidad, ubicación y ofertas ya solicitadas.
 - Ver las solicitudes asignadas por el profesor: consultar estado e historial de solicitudes realizadas por el profesor.
 - Participar en tutorías con el profesor asignado: interactuar en el proceso de supervisión de las prácticas.
- Funcionalidades del profesor
 - Buscar y solicitar ofertas para los alumnos asignados.
 - Supervisar tutorías: gestionar tutorías con los alumnos asignados durante las prácticas. Puede programar eventos o reuniones con el alumno asignado.
 - Ver las solicitudes.
- Funcionalidades de la empresa
 - Publicar ofertas de prácticas.
 - Gestionar ofertas publicadas (editar y borrar).
 - Ver solicitudes recibidas por los profesores.
 - Aceptar o rechazar solicitudes.
- Funcionalidades del sistema
 - Enviar notificaciones a cada entidad.

3.3 DISEÑO DEL PROGRAMA

La arquitectura del sistema se ha implementado a partir de la arquitectura Modelo-Vista-Controlador (MVC), es decir, un sistema que permite separar la lógica de negocio para poder conseguir una arquitectura modular y tener mejor ordenado el proyecto. Se ha conseguido esta arquitectura gracias a Laravel, debido a sus útiles herramientas para desarrollar el proyecto mediante dicha arquitectura. Esta arquitectura se ha compuesto en tres capas:

- Capa de presentación (frontend): aquella capa que va a dibujar y definir cómo se va a ver el sistema mediante componentes como botones, desplegables, campos de texto, formularios, etc. En este caso, se ha implementado mediante la utilización de React, que ha permitido crear componentes y reutilizarlos para las distintas páginas del

sistema, aparte se ha aplicado estilos mediante hojas de estilo CSS. Por ello, se ha creado la siguiente estructura:



Esta estructura se compone por:

- `src/public/resources/views`: contiene ficheros `.blade` que son aquellos archivos que se han utilizado para llamar a los componentes React.

- src/public/resources/css; hojas de estilo CSS de todos los componentes del sistema.
- src/public/resources/js: ficheros jsx pertenecientes a React para crear los componentes. Se compone de los siguientes directorios:
 - auth: componentes y funcionalidades de manera dinámica de los formularios de registro de cada rol.
 - feed: componentes y funcionalidades futuras para la feed de la aplicación (publicaciones, actividades de las empresas...).
 - header: componentes y funcionalidades dinámicas para los encabezados.
 - offers: componentes y funcionalidades dinámicas de la sección de ofertas.
 - requests: componentes y funcionalidades dinámicas de las solicitudes.
 - shared: otros componentes y funcionalidades.
 - tutorial: componentes y funcionalidades dinámicas de la sección de tutorías

Cada uno de los componentes han sido enrutados dependiendo de cada rol de usuario mediante el token de autenticación llamada en la API. Por otro lado, se ha utilizado peticiones AJAX para llamar a la API y cargar de manera asíncrona funcionalidades de la página de manera dinámica.

```

const logout = () => {
  const token = localStorage.getItem("api_token");
  let url = "/api/usuario/logout";

  fetch(url, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${token}`,
      "Content-Type": "application/json",
    },
  });
  localStorage.removeItem("api_token");
  localStorage.removeItem("user");
  window.location.href = "/login";
};

const fetchNotificaciones = () => {
  const token = localStorage.getItem("api_token");

  let url = "/api/usuario/notificacion";

  fetch(url, {
    method: "GET",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
      Accept: "application/json",
    },
  })
    .then((res) => res.json())
    .then((data) => {
      if (data.success) {
        setNotificaciones(data.notificaciones);
      }
    });
};

```

Imagen representativa de funcionalidad dinámica aplicada con peticiones AJAX

- Capa de Lógica de Negocio (backend): es aquella capa que interactúa entre la interfaz de usuario (capa de presentación) y la base de datos (capa de datos), es decir, actúa como intermediario entre las solicitudes del usuario y la base de datos. Esta capa incorpora validaciones, reglas y procesos para definir la creación, el almacenamiento y cambios de datos para garantizar el correcto comportamiento del sistema. Para ello, se ha implementado una API para realizar peticiones a la base de datos y recuperar, crear, editar o eliminar datos. La API se ha compuesto de los siguientes endpoints:

ENDPOINTS USUARIO

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/usuario	Listar todos los usuarios	NO	NO
GET	/api/usuario/{id}	Listar un usuario por su id	NO	id
POST	/api/usuario/login	Iniciar sesión	NO	email, contraseña
POST	/api/usuario/logout	Cerrar sesión	SI	NO
GET	/api/usuario/notificacion	Listar notificaciones de los usuarios	SI	NO

Endpoint	Respuesta
/api/usuario	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Usuario", "usuarios": [{ "id": 1, "nombre": "Consuelo", "apellidos": "Moya Gutierrez", "contrasena": "\$2y\$12\$K0Ru6odbyDd1h0XNSlbnys9Scw1y9vULvS.LX6TwDwsatZ6Ma.HG", "email": "consuelo@gmail.com", "activo": 1, "fecha_registro": "2026-05-01 10:03:00" }, { "id": 2, "nombre": "Iker", "apellidos": "Magro Juárez", "contrasena": "\$2y\$12\$85ff.P5vPjphR7xMWDPKSuE6/8RsmMgpXh083qu70trVooqcEY0ty", "email": "imagrojuarez@gmail.com", "activo": 1, "fecha_registro": "2026-05-01 10:04:09" }] }</pre>
/api/usuario/{id}	<pre>{ "response": 200, "success": true, "status": "ok", "usuario": { "id": 1, "nombre": "Consuelo", "apellidos": "Moya Gutierrez", "contrasena": "\$2y\$12\$K0Ru6odbyDd1h0XNSlbnys9Scw1y9vULvS.LX6TwDwsatZ6Ma.HG", "email": "consuelo@gmail.com", "activo": 1, "fecha_registro": "2026-05-01 10:03:00" } }</pre>
/api/usuario/login	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Inicio de sesión correcto.", "api_token": "69PitPxxvgG2sNiJp8Zbaf8BWSzpikmJUKMTYgWVUcGD7sNzuVHXiE1e74Gbf", "nombre": "Iker", "id_rol": 1, "rol": "alumno" }</pre>

/api/usuario/logout	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Sesión cerrada correctamente." }</pre>
/api/usuario/notificacion	<pre>{ "response": 200, "success": true, "message": "notificaciones", "notificaciones": [{ "id": "11cc5d42-ede6-42de-b3b8-0de5b974bb08", "type": "App\\Notifications\\Notificacion", "notifiable_type": "App\\Models\\Usuario", "notifiable_id": 2, "data": { "mensaje": "Tienes una nueva solicitud.", "datos": { "solicitud_id": 2 } }, "read_at": null, "created_at": "2026-05-01T10:52:48.000000Z", "updated_at": "2026-05-01T10:52:48.000000Z" }] }</pre>

ENDPOINTS ALUMNO

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/alumno	Listar todos los alumno	NO	NO
GET	/api/alumno/{id}	Listar un alumno por su id	NO	id
POST	/api/alumno/register	Registrar un alumno	NO	nombre, apellidos, contrasena, email, dni, telefono, id_profesor, id_centro, id_grado, fecha_nacimiento, curso, dni, cv
GET	/api/alumnos/profesor	Listar los alumnos asignados por el profesor	SI	NO

Endpoint	Respuesta
/api/alumno	<pre> { "response": 200, "success": true, "status": "ok", "message": "Alumno", "alumnos": [{ "id": 1, "id_usuario": 2, "id_profesor": 1, "id_centro": 1, "id_grado": 1, "curso": "2º", "dni": "54285820R", "cv": "cv/zUI11gdg4qgSNABj6LBAbszViUScGB3zkhPXZJLW.pdf", "disponibilidad": 1, "usuario": { "id": 2, "nombre": "Iker", "apellidos": "Magro Juárez", "contrasena": "\$2y\$12\$B6ff.PSVpJphR7xWDPKSuEG/0RsmMgpXh883qu70trVooqeEY8ty", "email": "imagrojuarez@gmail.com", "telefono": "661153989", "activo": 1, "fecha_registro": "2026-05-01 10:04:09", "api_token": "69PitPxvgG2sNiJp8Zbaf8BWSzpikmJUKMTYgWVUcG07sNzuVHXiE1e74GbF" } }] } </pre>
/api/alumno/{id}	<pre> { "response": 200, "success": true, "status": "ok", "alumno": { "id": 1, "id_usuario": 2, "id_centro": 1, "id_grado": 1, "curso": "2º", "dni": "54285820R", "cv": "cv/zUI11gdg4qgSNABj6LBAbszViUScGB3zkhPXZJLW.pdf", "disponibilidad": 1 } } </pre>
/api/alumno/register	<pre> { "response": 201, "success": true, "status": "ok", "message": "Cuenta de alumno creada correctamente." } </pre>

/api/alumnos/profesor	<pre>{ "response": 200, "success": true, "status": "ok", "alumnos": [{ "id": 1, "id_usuario": 2, "id_centro": 1, "id_grado": 1, "curso": "2º", "dni": "54285828R", "cv": "cv/zUI1Igd4qgSNABj6LBAbszViUScGB3zkhPXZJLW.pdf", "disponibilidad": 1, "usuario": { "id": 2, "nombre": "Iker", "apellidos": "Magro Juárez", "contrasena": "\$2y\$12\$B5ff.PSvPjphR7xWDPKSuE6/0RsmMgpXh883qu70trVqoqeEY8ty", "email": "imagrojuarez@gmail.com", "telefono": "661153989", "activo": 1, "fecha_registro": "2026-05-01 10:04:09", "api_token": "69PitPxvgG2sNiJp8Zbaf8BWSzpikmjUKMTYgWUcG07sNzuVHXiE1e746bf" }, "grado": { "id": 1, "nombre": "Desarrollo de Aplicaciones Web", "tipo": "Grado superior", "familia_profesional": "Informática y Comunicaciones", "codigo_grado": "IFCS03" } }], }</pre>
-----------------------	--

ENDPOINTS EMPRESA

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/empresa	Listar todas las empresas	NO	NO
GET	/api/empresa/{id}	Listar una empresa por su id	NO	id
POST	/api/empresa/registro	Registrar una empresa	NO	nombre, contrasena, email, telefono, id_sector, nif, descripcion, direccion, web
GET	/api/empresa/aceptado/{id_alumno}	Listar todas las empresas que ha aceptado la solicitud a los alumnos	SI	id_alumno

Endpoint	Respuesta
/api/empresa	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Empresa", "empresas": [{ "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "68639938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaría encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, Maria Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "S2y\$12\$rr8ysrdxa02B10A7i8p64uQ9lt2Pc.olgKrr2mKmjTjooCK6NIPS", "email": "microsoft@gmail.com", "telefono": "664567895", "activo": 1, "fecha_registro": "2026-05-01 10:06:08", "api_token": "H2M2n2Jgs9QVEYkHo7ndPwBztQHEnYVBqrpzhoPc2SRFSaJq2omAKYynQH" } }] }</pre>
/api/empresa/{id}	<pre>{ "response": 200, "success": true, "status": "ok", "empresa": { "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "68639938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaría encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, Maria Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1 } }</pre>
/api/empresa/register	<pre>{ "response": 201, "success": true, "status": "ok", "message": "Cuenta de empresa creada correctamente." }</pre>
/api/empresa/aceptado/{id_alumno}	<pre>{ "response": 200, "success": true, "status": "ok", "empresas": [{ "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "68639938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaría encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, Maria Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "S2y\$12\$rr8ysrdxa02B10A7i8p64uQ9lt2Pc.olgKrr2mKmjTjooCK6NIPS", "email": "microsoft@gmail.com", "telefono": "664567895", "activo": 1, "fecha_registro": "2026-05-01 10:06:08", "api_token": "H2M2n2Jgs9QVEYkHo7ndPwBztQHEnYVBqrpzhoPc2SRFSaJq2omAKYynQH" } }] }</pre>

ENDPOINTS PROFESOR

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/profesor	Listar todas los profesores	NO	NO
GET	/api/profesor/{id}	Listar un profesor por su id	NO	id
POST	/api/profesor/register	Registrar un profesor	NO	nombre, contrasena, email, telefono, id_centro, id_grado, id_departamento, dni
GET	/api/profesor/centro/{id_centro}	Listar todos los profesores por el centro educativo	NO	id_centro

Endpoint	Respuesta
/api/profesor	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Profesor", "alumnos": [{ "id": 1, "id_usuario": 1, "id_centro": 1, "id_grado": 1, "id_departamento": 1, "dni": "541988678", "usuario": { "id": 1, "nombre": "Consuelo", "apellidos": "Moya Gutierrez", "contrasena": "\$2y\$12\$K0Ru6odbyDd1h0XNS1bnyes9Scw1y9vULvS.LX6TwDwsatZ6Ma.HG", "email": "consuelo@gmail.com", "telefono": "663456789", "activo": 1, "fecha_registro": "2026-05-01 10:03:08", "api_token": "PEUQxeXB1nnvcLJHQB33U8CtohhRPNFY9U9T7vIooyuNktG06nFqM0su8H9c" } }] }</pre>

/api/profesor/{id}	<pre>{ "response": 200, "success": true, "status": "ok", "profesor": { "id": 1, "id_usuario": 1, "id_centro": 1, "id_grado": 1, "id_departamento": 1, "dni": "54198867B" } }</pre>
/api/profesor/register	<pre>{ "response": 201, "success": true, "status": "ok", "message": "Cuenta de profesor creada correctamente." }</pre>
/api/profesor/centro/{id_centro}	<pre>{ "response": 200, "success": true, "status": "ok", "profesores": [{ "id": 1, "id_usuario": 1, "id_centro": 1, "id_grado": 1, "id_departamento": 1, "dni": "54198867B", "usuario": { "id": 1, "nombre": "Consuelo", "apellidos": "Moya Gutierrez", "contrasena": "\$2y\$12\$sk0Ru6odbyDd1h0XNS1bnyes9Scw1y9vULvS.LX6TwDwsatZ6Ma.HG", "email": "consuelo@gmail.com", "telefono": "663456789", "activo": 1, "fecha_registro": "2026-05-01 10:03:08", "api_token": "PEUQxeXB1nnvcL3HQBJ3U8CtohhRPhFY9U9T7vIooyuNkTG6nFqMDsu8H9c" } }], }</pre>

ENDPOINTS OFERTAS

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/oferta	Listar todas las ofertas	NO	NO
GET	/api/oferta/{id}	Listar una oferta por su id	NO	id
GET	/api/oferta/filtrar	Filtrar ofertas por modalidad, sector y/o título	NO	modalidad, id_sector, titulo
GET	/api/oferta/empr esa	Listar todas las ofertas propias de la empresa logueada	SI	NO
POST	/api/oferta/crear	Crear una nueva oferta de la empresa logueada	SI	titulo, descripcion, requisitos, modalidad, estado
DELETE	/api/oferta/{id}/e liminar	Eliminar oferta de la empresa logueada	SI	id

Endpoint	Respuesta
/api/oferta	<pre> { "response": 200, "success": true, "status": "ok", "message": "Oferta", "alumnos": [{ "id": 1, "id_empresa": 1, "titulo": "Desarrollador de Java/Springboot", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Java y Springboot", "requisitos": "Java, Springboot", "modalidad": "Presencial", "fecha_publicacion": "2026-06-01", "estado": "Abierta", "empresa": { "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "68639938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaria encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, Maria Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "S2y\$128xrByardxa02B10A718p64uQ9Lt2Pc.oIGKxz2wKwojtjooCK6NIPS", "email": "microsoft@gmail.com", "telefono": "464607090", "activo": 1, "fecha_registro": "2026-06-01 10:06:05", "api_token": "MCN2n2IG649QVEyKHo7ndPw@rtQHEnYVBQzprhosPc2SRFS0ja2onAKYynQN" } } }], }</pre>

/api/oferta/{id}	<pre>{ "response": 200, "success": true, "status": "ok", "oferta": { "id": 1, "id_empresa": 1, "titulo": "Desarrollador de Java/Springboot", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Java y Springboot", "requisitos": "Java, Springboot", "modalidad": "Presencial", "fecha_publicacion": "2026-05-01", "estado": "Abierta" } }</pre>
/api/oferta/filtrar	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Oferta", "ofertas": [{ "id": 1, "id_empresa": 1, "titulo": "Desarrollador de Java/Springboot", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Java y Springboot", "requisitos": "Java, Springboot", "modalidad": "Presencial", "fecha_publicacion": "2026-05-01", "estado": "Abierta", "eliminado": 0, "empresa": { "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "60639930M", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaria encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, Maria Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "\$2y\$12\$zrr0yrdxa02810A718p64uQ9L2Pc.oLgKxz2wMojtjooCK6N1PS", "email": "microsoft@gmail.com", "telefono": "664567895", "activo": 1, "fecha_registro": "2026-05-01 10:06:05", "api_token": "H2KZn21gs9QVEYkxh7ndPw@rtQHEnVVBqzptxosPc2SRF3oJozonAKVynqN" } } }] }</pre>
/api/oferta/empresa	<pre>{ "response": 200, "success": true, "status": "ok", "ofertas": [{ "id": 1, "id_empresa": 1, "titulo": "Desarrollador de Java/Springboot", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Java y Springboot", "requisitos": "Java, Springboot", "modalidad": "Presencial", "fecha_publicacion": "2026-05-01", "estado": "Abierta", "eliminado": 0 }, { "id": 2, "id_empresa": 1, "titulo": "Desarrollador de Python", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Python.", "requisitos": "Python", "modalidad": "Remoto", "fecha_publicacion": "2026-05-01", "estado": "Abierta", "eliminado": 0 }] }</pre>
/api/oferta/crear	<pre>{ "response": 201, "success": true, "status": "ok", "message": "Se ha creado la oferta correctamente." }</pre>

/api/oferta/{id}/eliminar	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Oferta eliminada correctamente." }</pre>
---------------------------	---

ENDPOINTS SOLICITUD

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/solicitud/profesor	Listar todas las solicitudes enviadas por el profesor logueado	SI	NO
GET	/api/solicitud/alumno	Listar todas las solicitudes enviadas por el profesor asignado por el alumno logueado	SI	NO
GET	/api/solicitud/empresa	Listar todas las solicitudes enviadas por el profesor para la empresa logueada	SI	NO
POST	/api/solicitud/profesor/crear	Enviar una o varias solicitudes a la oferta por el profesor logueado	SI	id_oferta, id_empresa, alumnos
PUT	/api/solicitud/{id}/actualizar	Actualizar estado de la solicitud por la empresa logueada	SI	estado
GET	/api/solicitud/empresa/alumno/{id}/aceptado	Listar empresas del alumno que tiene aceptado la solicitud	SI	id

Endpoint	Respuesta
/api/solicitud/profesor	<pre>{ "response": 200, "success": true, "status": "OK", "solicitud": [{ "id": 1, "id_oferta": 1, "id_alumno": 1, "id_profesor": 1, "fecha_solicitud": "2026-06-01 10:08:37", "estado": "Aceptada", "oferta": { "id": 1, "id_empresa": 1, "titulo": "Desarrollador de Java/Springboot", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Java y Springboot", "requisitos": "Java, Springboot", "modalidad": "Presencial", "fecha_publicacion": "2026-06-01", "estado": "Abierta", "eliminado": 0, "empresa": { "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "68639938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaria encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, María Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "S2y\$12zr8yaxda02B10A718p64uQ9lt2Pc.olgKxz2wMojtjooCK6MIPS", "email": "microsoft@gmail.com", "telefono": "664567895", </pre>
/api/solicitud/alumno	<pre>{ "response": 200, "success": true, "status": "OK", "solicitud": [{ "id": 1, "id_oferta": 1, "id_alumno": 1, "id_profesor": 1, "fecha_solicitud": "2026-06-01 10:08:37", "estado": "Aceptada", "oferta": { "id": 1, "id_empresa": 1, "titulo": "Desarrollador de Java/Springboot", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Java y Springboot", "requisitos": "Java, Springboot", "modalidad": "Presencial", "fecha_publicacion": "2026-06-01", "estado": "Abierta", "eliminado": 0, "empresa": { "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "68639938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaria encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, María Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "S2y\$12zr8yaxda02B10A718p64uQ9lt2Pc.olgKxz2wMojtjooCK6MIPS", "email": "microsoft@gmail.com", "telefono": "664567895", </pre>
/api/solicitud/empresa	<pre>{ "response": 200, "success": true, "status": "OK", "solicitud": [{ "id": 1, "id_oferta": 1, "id_alumno": 1, "id_empresa": 1, "id_profesor": 1, "fecha_solicitud": "2026-06-01 10:08:37", "estado": "Aceptada", "eliminado": 0, "oferta": { "id": 1, "id_empresa": 1, "titulo": "Desarrollador de Java/Springboot", "descripcion": "Buscamos gente de prácticas que tenga experiencia con Java y Springboot", "requisitos": "Java, Springboot", "modalidad": "Presencial", "fecha_publicacion": "2026-06-01", "estado": "Abierta", "eliminado": 0, "empresa": { "id": 1, "id_usuario": 3, "id_sector": 1, "nif": "68639938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaria encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, María Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "S2y\$12zr8yaxda02B10A718p64uQ9lt2Pc.olgKxz2wMojtjooCK6MIPS", </pre>

/api/solicitud/profesor/crear	<pre>{ "response": 201, "success": true, "status": "ok", "message": "Solicitud enviada correctamente." }</pre>
/api/solicitud/{id}/actualizar	<pre>{ "response": 200, "success": true, "status": "ok", "message": "La solicitud se ha actualizado correctamente." }</pre>
/api/solicitud/empresa/alumno/{id}/aceptado	<pre>{ "response": 200, "success": true, "status": "ok", "message": [{ "id": 1, "id_usuario": 3, "id_empresa": 1, "nif": "68679938H", "descripcion": "Hola somos el equipo de Microsoft.\r\nNos encantaria encontrar becarios profesionales de prácticas para nuestra empresa para que se desarrolle en nuestro entorno laboral.", "direccion": "Calle, Maria Curie, 3", "web": "https://www.microsoft.com/es-es", "activo": 1, "usuario": { "id": 3, "nombre": "Microsoft", "apellidos": null, "contrasena": "S2y\$128r8yazda02B10A718p4uQ91t2Pc.o1gKxz2w0mjtjooCKM1P9", "email": "microsoft@gmail.com", "telefono": "664567895", "activo": 1, "fecha_registro": "2026-06-01 10:06:06", "api_token": "H2M2n21gs9QVEyho7ndPw8rtQhEnVW8qrphosPc25RFSojq2omAKYynQN" }, "id_alumno": 1 }] }</pre>

ENDPOINTS TUTORÍAS

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/tutoria/alumno	Listar todas las tutorías asignadas al alumno logueado	SI	NO
GET	/api/tutoria/profesor	Listar todas las tutorías del profesor logueado	SI	NO
POST	/api/tutoria/crear	Crear tutoría por el profesor logueado	SI	id_alumno, id_profesor, id_empresa, fecha_inicio, fecha_fin, estado
PUT	/api/tutoria/{id}/actualizar	Actualizar tutoría por el profesor logueado	NO	id, fecha_inicio, fecha_fin, estado
PUT	/api/tutoria/{id}/eliminar	Eliminar tutoría por el profesor logueado	NO	id

Endpoint	Respuesta
/api/tutoria/profesor	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Alumno", "Tutorias": [{ "id": 5, "id_alumno": 1, "id_profesor": 1, "id_empresa": 1, "fecha_inicio": "2026-05-05 16:00:00", "fecha_fin": "2026-05-05 16:30:00", "estado": "Activa", "alumno": { "id": 1, "id_usuario": 2, "id_profesor": 1, "id_centro": 1, "id_grado": 1, "fecha_nacimiento": "2005-02-28", "curso": "2º", "dni": "54285820R", "cv": "cv/zUI1Igdg4ggSNABj6LBAbszViUScGB3zkhPXZJLW.pdf", "disponibilidad": 1, "eliminado": 0, "usuario": { "id": 2, "nombre": "Iker", "apellidos": "Magro Juárez", "contrasena": "\$2y\$12\$B5ff.PSvPjphR7xwWDPKSuEG/0RemMgpXh883qu70trVqoqoEY6ty", "email": "imagrojuarez@gmail.com", "telefono": "661153989", "activo": 1, "fecha_registro": "2026-05-01 10:04:09", "api_token": "69PitPxvgG2sN1jp8ZbaF8BWSzpikmJUKNTYgWVUcGD7sNzuVHX1E1e74Gbf" } } }], }</pre>
/api/tutoria/alumno	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Alumno", "Tutorias": [{ "id": 5, "id_alumno": 1, "id_profesor": 1, "id_empresa": 1, "fecha_inicio": "2026-05-05 16:00:00", "fecha_fin": "2026-05-05 16:30:00", "estado": "Activa", "profesor": { "id": 1, "id_usuario": 1, "id_centro": 1, "id_grado": 1, "id_departamento": 1, "dni": "54198867B", "eliminado": 0, "usuario": { "id": 1, "nombre": "Consuelo", "apellidos": "Moya Gutierrez", "contrasena": "\$2y\$12\$KDRu6odbyDd1h0XNS1bnyes9Scw1y9vULvS.LX6Tw0wsatZ6Ma.HG", "email": "consuelo@gmail.com", "telefono": "663456789", "activo": 1, "fecha_registro": "2026-05-01 10:03:00", "api_token": "PEUQxeXB1nnvcLJHQ8J3U8CtohhRPNFY9U9T7vIooyuNktG86nFqMDsu8H9c" } } }], }</pre>
/api/tutoria/crear	<pre>{ "response": 201, "success": true, "status": "ok", "message": "Se ha creado la tutoría correctamente." }</pre>

/api/tutoria/{id}/actualizar	<pre>{ "response": 200, "success": true, "status": "ok", "message": "La tutoria se ha actualizado correctamente." }</pre>
/api/solicitud/{id}/eliminar	<pre>{ "response": 200, "success": true, "status": "ok", "message": "La tutoria ha sido eliminado correctamente." }</pre>

ENDPOINTS AUXILIARES

Método	Endpoint	Descripción	Autenticación	Parámetros
GET	/api/centro	Listar centros educativos	NO	NO
POST	/api/centro/crear	Crear centro educativo	NO	nombre, localidad, provincia, codigo_centro
GET	/api/grado	Listar grados formativos	NO	NO
POST	/api/grado/crear	Crear grado	NO	nombre, tipo, familia_profesional, codigo_grado
GET	/api/sector	Listar sectores empresariales	NO	NO
POST	/api/sector/crear	Crear sectores empresariales	NO	nombre
GET	/api/departamen to	Listar departamentos de centros educativos	NO	NO
POST	/api/departamen to/crear	Crear departamento de centro educativo	NO	nombre

Endpoint	Respuesta
/api/centro	<pre> { "response": 200, "success": true, "status": "ok", "message": "Centro educativo", "centros": [{ "id": 1, "nombre": "IES Laguna de Joaztel", "localidad": "Getafe", "provincia": "Madrid", "codigo_centro": "28039827" }] } </pre>
/api/centro/crear	<pre> { "response": 201, "success": true, "status": "ok", "message": "Se ha creado el centro educativo correctamente." } </pre>
/api/grado	<pre> { "response": 200, "success": true, "status": "ok", "message": "Grado", "grados": [{ "id": 1, "nombre": "Desarrollo de Aplicaciones Web", "tipo": "Grado superior", "familia_profesional": "Informática y Comunicaciones", "codigo_grado": "IFCS03" }, { "id": 2, "nombre": "Desarrollo de Aplicaciones Multiplataforma", "tipo": "Grado superior", "familia_profesional": "Informática y Comunicaciones", "codigo_grado": "IFCS04" }] } </pre>
/api/grado/crear	<pre> { "response": 201, "success": true, "status": "ok", "message": "Se ha creado el grado correctamente." } </pre>

/api/sector	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Sector", "sectores": [{ "id": 1, "nombre": "Informática y Telecomunicaciones" }, { "id": 2, "nombre": "Marketing" }] }</pre>
/api/sector/crear	<pre>{ "response": 201, "success": true, "status": "ok", "message": "Se ha creado el sector correctamente." }</pre>
/api/departamento	<pre>{ "response": 200, "success": true, "status": "ok", "message": "Departamento", "departamentos": [{ "id": 1, "nombre": "Informática y Comunicaciones" }] }</pre>
/api/departamento/crear	<pre>{ "response": 201, "success": true, "status": "ok", "message": "Se ha creado el departamento correctamente." }</pre>

- Capa de datos (base de datos): es aquella capa que se va a encargar de gestionar y almacenar la información para que pueda recibir la petición el usuario o el servidor. Para ello, se ha creado la base de datos a partir de las herramientas ofrecidas por Lavarel, es decir, el modelo (models) y las migraciones (migrations):
 - models: son las distintas clases para representar los modelos de datos de cada tabla para que reciba la petición el usuario y el servidor.

- migrations: son las distintas clases que permiten migrar tablas y relaciones a la base de datos (MySQL).

```

31 references | 0 implementations
class Usuario extends Authenticatable
{
    use HasFactory, HasApiTokens, Notifiable;
    0 references
    protected $table = 'usuario';

    0 references
    public $timestamps = false;
    0 references
    protected $fillable = [
        'nombre',
        'apellidos',
        'contrasena',
        'email',
        'telefono',
        'activo',
        'fecha_registro',
        'api_token'
    ];

    1 reference | 0 overrides
    public function alumno()
    {
        return $this->hasOne(Alumno::class, 'id_usuario');
    }

    1 reference | 0 overrides
    public function profesor()
    {
        return $this->hasOne(Profesor::class, 'id_usuario');
    }

    1 reference | 0 overrides
    public function empresa()
    {
        return $this->hasOne(Empresa::class, 'id_usuario');
    }
}

```

Imagen representativa de la clase Usuario que se ha utilizado para recibir el modelo de datos de la tabla usuario en la base de datos

```

return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('usuario', function (Blueprint $table) {
            $table->id();
            $table->string('nombre', 50)->nullable(false);
            $table->string('apellidos', 100)->nullable();
            $table->string('contrasena', 255)->nullable(false);
            $table->string('email', 100)->nullable(false)->unique();
            $table->string('telefono', 20)->nullable(false);
            $table->boolean('activo')->default(1);
            $table->timestamp('fecha_registro')->useCurrent();
            $table->string('api_token', 80)->unique()->nullable();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('usuario');
    }
};

```

Imagen representativa de la migración de la tabla usuario

COMANDOS UTILIZADOS

Se ha utilizado comandos para manipular el proyecto realizado con Laravel mediante el despliegue en local en Docker Compose:

Comando	Explicación
<code>docker compose up -d</code>	Levantar contenedores en modo desarrollador
<code>docker compose down</code>	Apagar contenedores
<code>docker compose exec app npm run dev --host</code>	Levantar contenedor de Node.js en modo desarrollador para compilar los componentes React
<code>docker compose exec mysql mysql -u -root -p</code>	Ejecutar y abrir el contenedor de base de datos para visualizar la base de datos
<code>docker compose exec app php artisan migrate</code>	Ejecutar dentro del contenedor la migración completa de las tablas a la base de datos
<code>docker compose exec app php artisan migrate:fresh</code>	Refrescar migración completa de las tablas a la base de datos (elimina los registros y actualiza los cambios en las tablas)
<code>docker compose exec app php artisan test tests/Unit</code>	Ejecutar pruebas unitarias dentro del proyecto de Laravel

3.4 CODIFICACIÓN

La implementación de ConectaFP se ha realizado principalmente en PHP, utilizando el framework Laravel para facilitar la creación del proyecto mediante la arquitectura MVC y API RestFul. La gestión del proyecto y las dependencias se realizan a partir del composer y node.js (npm).

Lenguajes y herramientas utilizadas:

- Lenguajes principales: PHP, CSS y JSX (React).
- Framework: Laravel.
- Persistencia de datos: migraciones a partir de Laravel.
- Base de datos: MySQL.
- Gestor de dependencias y build: Docker Compose (build), composer (Laravel) y node.js (dependencias para React).
- Pruebas: Postman (peticiones para probar la API).
- IDE: Visual Studio Code.

Estructura del código:

El código se ha organizado gracias a Laravel mediante la arquitectura MVC de la siguiente manera:

- src/app/http/controllers: lógica de negocio del sistema.
- src/app/models: modelo de datos.
- src/app/notifications: sistema de notificaciones facilitado con Laravel.
- src/database/migrations: migraciones de las tablas.
- src/resources/css: hojas de estilo CSS.
- src/resources/js: componentes React.
- src/resources/views: plantillas .blade donde se van a llamar los componentes React.
- src/routes: rutas de la API y rutas de las distintas páginas del sitio web.

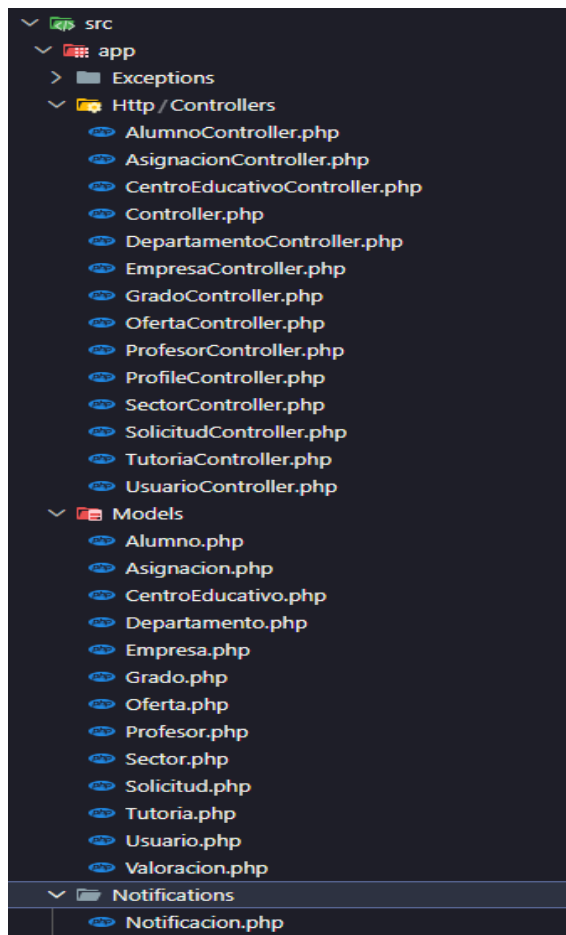


Imagen representativa de la estructura de la capa de datos y la capa de la lógica de negocio

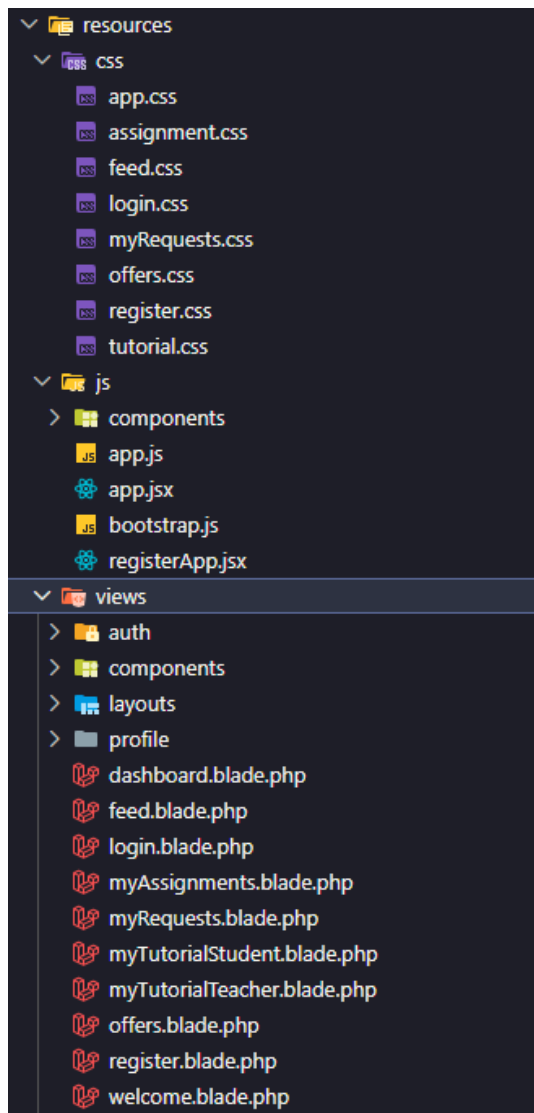


Imagen representativa de la estructura de la capa de presentación

3.5 EJECUCIÓN DE PRUEBAS

Para asegurar que la aplicación web funciona de manera correcta, se ha realizado una serie de pruebas para la API mediante validaciones a partir de los códigos de respuestas HTTP:

Código	Respuesta	Descripción
200	OK	Operación exitosa
201	Created	Recurso creado correctamente
401	Unauthorized	No está autorizado (token no válido o expirado)
404	Not Found	Recurso no encontrado
422	Validation Error	Error en la validación de datos
500	Server Error	Error interno del servidor

A parte de realizar pruebas en Postman se han implementado pruebas unitarias en la API, gracias a las herramientas que ofrece Laravel y poder asegurar que el software funciona correctamente. Para ello, se ha seguido los siguientes pasos:

1. Creación de las clases de los tests unitarios:

```
docker compose exec app php artisan make:test nombreArchivo.php --unit
```

2. Ejecución de los tests unitarios:

- a. Ejecución de un solo test:

```
docker compose exec app php artisan test tests/Unit/nombreArchivo.php
```

- b. Ejecución de todos los tests:

```
docker compose exec app php artisan test tests/Unit/
```

```

PASS Tests\Unit\UsuarioApiTest
✓ can list all users
✓ can get user by id
✓ user login with valid credentials
✓ user login with invalid credentials

Tests: 75 passed (426 assertions)
Duration: 4.46s

```

Imagen representativa de la ejecución de los tests unitarios

3.6 VERIFICACIÓN

La verificación del sistema ConectaFP se ha realizado mediante pruebas en la API a partir de Postman, pruebas unitarias y validación manual, asegurando que cada requisito funcional esté correctamente implementado y funcionando según lo esperado.

3.7 MANTENIMIENTO

El sitio web ha sido diseñado pensando en la escalabilidad y modularidad, lo que facilita su mantenimiento y futuras ampliaciones. A continuación se detallan los aspectos clave:

- Escalabilidad
 - La arquitectura en capas y el uso del patrón MVC permiten añadir nuevas funcionalidades, por ejemplo: historial de tutorías y ofertas eliminadas para ser recuperadas, sistema de mensajería, etc, sin afectar el núcleo del sistema.
- Modularidad
 - El código se ha organizado en distintos directorios bien diferenciados gracias a Laravel para facilitar la localización y modificación de componentes específicos.
- Futuras versiones y ampliaciones
 - Autenticación externa (Google).
 - Feed para que las empresas, alumnos y profesores puedan publicar publicaciones, actividades desarrolladas en las empresas o valoraciones de las empresas y alumnos.
 - Sistema de mensajería y comunicación en la sección de tutoría (chat).
 - Gestión de perfil (seguidores, estudios...).

6. DESARROLLO Y EJECUCIÓN DEL PROYECTO

a) Análisis y definición de requisitos:

El proyecto comenzó con identificar una necesidad al usuario: crear una aplicación web o red social que permitiera a los alumnos encontrar de manera sencilla ofertas de prácticas de Formación Profesional en España. Para ello, se elaboró un listado de requisitos funcionales y no funcionales, así como los principales casos de uso para asegurar una solución al problema.

b) Diseño de la arquitectura y la base de datos:

Se optó por una arquitectura MVC (Modelo-Vista-Controlador) para garantizar la separación de la lógica de negocio y facilitar el mantenimiento. Se ha diseñado la estructura de directorias y migraciones, definiendo las tablas. Se ha elegido tanto Laravel como React para facilitar el desarrollo web.

c) Integración de la interfaz gráfica:

Se ha diseñado y aplicado estilos CSS personalizados e imágenes para mejorar la accesibilidad y la experiencia visual.

d) Pruebas y validación:

Se han desarrollado pruebas en Postman para comprobar que funciona cada API y se han realizado pruebas manuales para asegurar la robustez y la experiencia de usuario.

e) Documentación

Se ha preparado la documentación técnica y el README, detallando la estructura, requisitos y modo de uso.

f) Toma de decisiones

- Decisiones clave:
 - Uso de Laravel y MVC: garantizar una arquitectura limpia y ordenada.
 - React: reutilizar componentes según los roles de los usuarios.
- Dificultades superadas:
 - Validaciones y mensajes de error: se ha dedicado tiempo a asegurar que los mensajes fueran claros y útiles para el usuario.
 - Comandos: se ha dedicado tiempo a entender y a comprender el uso de los comandos durante el proyecto.
 - Modelo entidad-relación: se ha dedicado el tiempo a pensar el modelo entidad relación para que fuera comprensible y coherente.

7. CONCLUSIONES

El desarrollo de este proyecto me ha permitido adquirir conocimientos más avanzados de Laravel que antes no conocía, por ejemplo la integración con componentes React. Es importante mencionar que he aprendido a utilizar de mejor manera los contenedores Docker para utilizar los comandos de manera correcta.

Este proyecto me ha incentivado a explorar nuevas funcionalidades que podría implementar en un futuro cercano y lejano como implementar la misma aplicación en otras plataformas como móvil, a parte de seguir mejorando la interfaz gráfica.

8. LÍNEAS DE INVESTIGACIÓN Y DESARROLLO FUTURAS

Este sistema puede ofrecer nuevas funcionalidades y mejoras futuras:

- Versión móvil: adaptar la aplicación en otras plataformas como en móvil.
- Sistema de mensajería y comunicación en la sección de tutoría (chat).
- Autenticación externa (Google).
- Feed para que las empresas, alumnos y profesores puedan publicar publicaciones, actividades desarrolladas en las empresas o valoraciones de las empresas y alumnos.
- Gestión de perfil (seguidores, estudios...).

Estas mejoras y funcionalidades permitirán al sistema seguir evolucionando a una aplicación más completa y que cubra mucho más las necesidades de los usuarios.

9. BIBLIOGRAFÍA

<https://laravel.com/docs/13.x/queries> - Documentación de Laravel.

<https://github.com/danielmrdev/laravel-spanish-validator> - Documentación de la librería de laravel-spanish-validator.

<https://dashboard.render.com/> - Despliegue en entorno de producción en Render.

Claude Code - Apoyo para el despliegue y CSS.

10. EXTRAS

10.1 ANÁLISIS DE MERCADO

Los usuarios que utilizarían esta aplicación web sería aquellos alumnos que necesitan encontrar prácticas en empresas que las permitan hacer de manera rápida y sencilla. Por ello, las necesidades que se resuelven son:

- Ofertas: los alumnos pueden ver ofertas de prácticas de las empresas que están disponibles.
- Profesores: los profesores pueden solicitar ofertas a las empresas por los alumnos que tengan asignados.
- Gestión de solicitudes: la empresa se encargará de gestionar que alumnos son admitidos para las prácticas mediante la modificación de la solicitud enviada por el profesor del alumno asignado.
- Gestión de cuentas: se facilita al usuario elegir un rol (alumno, profesor o empresa) mediante un registro e inicio de sesión.

10.2 SEGURIDAD Y PROTECCIÓN DE DATOS

Las medidas de seguridad aplicadas en el software incluyen:

1. Validación de entradas del usuario.
 - a. En todos los formularios de la aplicación web, se valida los campos tanto en la API (servidor) como en los componentes React (cliente).
 - b. Se evita el registro de usuarios con el mismo correo electrónico existente en la API.

2. Gestión de contraseñas:

Las contraseñas han sido encriptadas en la API gracias a las herramientas que ofrece Laravel mediante la clase Facades/Hash:

```
'contrasena' => Hash::make($data['contrasena']),
```

Imagen representativa de un ejemplo para cifrar la contraseña mediante cifrado AES-256 gracias a la clase de Laravel: Facades/Hash

3. Autenticación de usuarios:

Las rutas de la aplicación web están protegidas mediante un token de autenticación por cada usuario gracias a la funcionalidad de Auth:Sanctum de Laravel. El token es guardado en la tabla Usuario y se recupera llamando al usuario mediante la clase Illuminate\Support\Facades\Auth.

```

    $user = Auth::user();
    if (!$user || !$user->profesor) {
        return response()->json([
            'response' => 401,
            'success' => false,
            'status' => 'error',
            'message' => 'No autenticado o el usuario no es profesor.'
        ], 401);
    }
    $idProfesor = $user->profesor->id;

```

Imagen representativa de la recuperación del usuario mediante el token a partir de la llamada del usuario a través de la clase Illuminate\Support\Facades\Auth

```

const logout = () => {
    const token = localStorage.getItem("api_token");
    let url = "/api/usuario/logout";

    fetch(url, {
        method: "POST",
        headers: {
            Authorization: `Bearer ${token}`,
            "Content-Type": "application/json",
        },
    });
    localStorage.removeItem("api_token");
    localStorage.removeItem("user");
    window.location.href = "/login";
};

```

Imagen representativa de la obtención del token de autenticación en el cliente gracias a localStorage en JavaScript

CONFIGURACIÓN DE LA AUTENTICACIÓN DE USUARIOS CON API:AUTH

La configuración de la autenticación del usuario mediante token se ha realizado mediante la modificación de la clase Auth.php para que guarde el token en la migración de la tabla usuario.

```

'guards' => [
  'web' => [
    'driver' => 'session',
    'provider' => 'users',
  ],
  'api' => [
    'driver' => 'token',
    'provider' => 'users',
    'hash' => false,
  ],
],

/*
|-----
| User Providers
|-----
|
| All authentication guards have a user provider, which defines how the
| users are actually retrieved out of your database or other storage
| system used by the application. Typically, Eloquent is utilized.
|
| If you have multiple user tables or models you may configure multiple
| providers to represent the model / table. These providers may then
| be assigned to any extra authentication guards you have defined.
|
| Supported: "database", "eloquent"
|
*/

'providers' => [
  'users' => [
    'driver' => 'eloquent',
    'model' => env('AUTH_MODEL', App\Models\Usuario::class),
  ],

  // 'users' => [
  //   'driver' => 'database',
  //   'table' => 'users',
  // ],
],

```

Imagen representativa de la configuración modificada para que funcione correctamente la autenticación por token a través de la tabla usuario

4. Manejo de errores y mensajes:

- Se muestran mensajes de error claros cuando ocurre un fallo en los formularios.
- Los errores de la API y de ejecución se capturan y no se exponen detalles técnicos en la vista.

5. Persistencia y acceso a datos:

- Las operaciones de la API se llaman a partir de peticiones AJAX en cliente, que carga mensajes personalizados para evitar inyecciones SQL.
- No se exponen datos sensibles como contraseñas en logs ni en la interfaz.

10.3 DESPLIEGUE

El despliegue de la aplicación web se ha desarrollado tanto en un entorno de desarrollo local como en un entorno de producción utilizando el Web Service de Render.

Antes de explicar cada despliegue, se ha utilizado las siguientes tecnologías:

Componente	Tecnología	Versión	Propósito
Contenedores	Docker	27.5.1	Visualización
Orquestación	Docker Compose	2.32.4	Gestión de múltiples contenedores (MySQL, Nginx y PHP)
Backend	PHP	8.2-fpm	Ejecución de Laravel
Framework Backend	Laravel	12	API REST y lógica de negocio
Base de datos (Local)	MySQL	8.0	Persistencia de datos para desarrollo en local
Base de datos (Producción)	PostgreSQL	Controlado por Render	Persistencia de datos en producción
Servidor web	Nginx	Alpine	Servidor HTTP
Compilación Frontend	Node.js	20.x	Compilación de los componentes React
Package Frontend	Vite		Empaquetado
Plataforma Cloud	Render		Hosting en producción

DESPLIEGUE EN ENTORNO LOCAL

1. Configuración de Docker Compose

El fichero docker-compose.yml se compone de tres servicios:

- PHP: Web Service de la aplicación mediante el desarrollo en PHP.
- Nginx: servidor web.
- MySQL: base de datos.

```
services:
  app:
    build:
      context: ./docker/apache
    container_name: conecta_fp_app
    ports:
      - "5173:5173"
    volumes:
      - ./src:/var/www
    working_dir: /var/www

  nginx:
    image: nginx:alpine
    container_name: laravel_nginx
    ports:
      - "8000:80"
    volumes:
      - ./src:/var/www
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - app

  mysql:
    image: mysql:8.0
    container_name: conecta_fp_db
    restart: always
    environment:
      MYSQL_DATABASE: conecta_fp
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - dbdata:/var/lib/mysql

volumes:
  dbdata:
```

Imagen representativa de las instrucciones de la orquestación de los contenedores en el fichero docker-compose.yml

2. Instalación local

- Paso 1. Iniciar contenedores Docker.

```
docker compose up -d
```

Este comando realiza lo siguiente:

- Descarga las imágenes necesarias.
- Construye el contenedor app según el Dockerfile.

- Inicia los tres servicios en segundo plano para su desarrollo en local.

```
[+] Running 4/4
✓ Network conectafp_default Created
✓ Container conecta_fp_app Started
✓ Container conecta_fp_db Started
✓ Container laravel_nginx Started
```

Imagen representativa de todos los contenedores iniciados

- Paso 2. Instalar dependencias del backend.

```
docker compose exec app composer install
```

Este comando instala todas las dependencias de PHP definidas en composer.json.

- Paso 3. Configurar variables de entorno y generar key para la aplicación de Laravel.

```
docker compose exec app php artisan key:generate
```

Este comando genera una key para que funcione correctamente la aplicación en Laravel

Para configurar las variables de entorno de la aplicación, se debe crear un fichero .env con la siguiente configuración:

```
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:PQJ0NX/90/9tym7qIVMGC131Y4sJ+pmWm22UB7pPzfw=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 APP_LOCALE=es
8 APP_FALLBACK_LOCALE=es
9 APP_FAKER_LOCALE=en_US
10
11 APP_MAINTENANCE_DRIVER=file
12 # APP_MAINTENANCE_STORE=database
13
14 # PHP_CLI_SERVER_WORKERS=4
15
16 BCRYPT_ROUNDS=12
17
18 LOG_CHANNEL=stack
19 LOG_STACK=single
20 LOG_DEPRECATED_CHANNEL=null
21 LOG_LEVEL=debug
22
23 DB_CONNECTION=mysql
24 DB_HOST=mysql
25 DB_PORT=3306
26 DB_DATABASE=conecta_fp
27 DB_USERNAME=root
28 DB_PASSWORD=root
```

- Paso 4. Ejecutar migraciones de base de datos.

```
docker compose exec app php artisan migrate
```

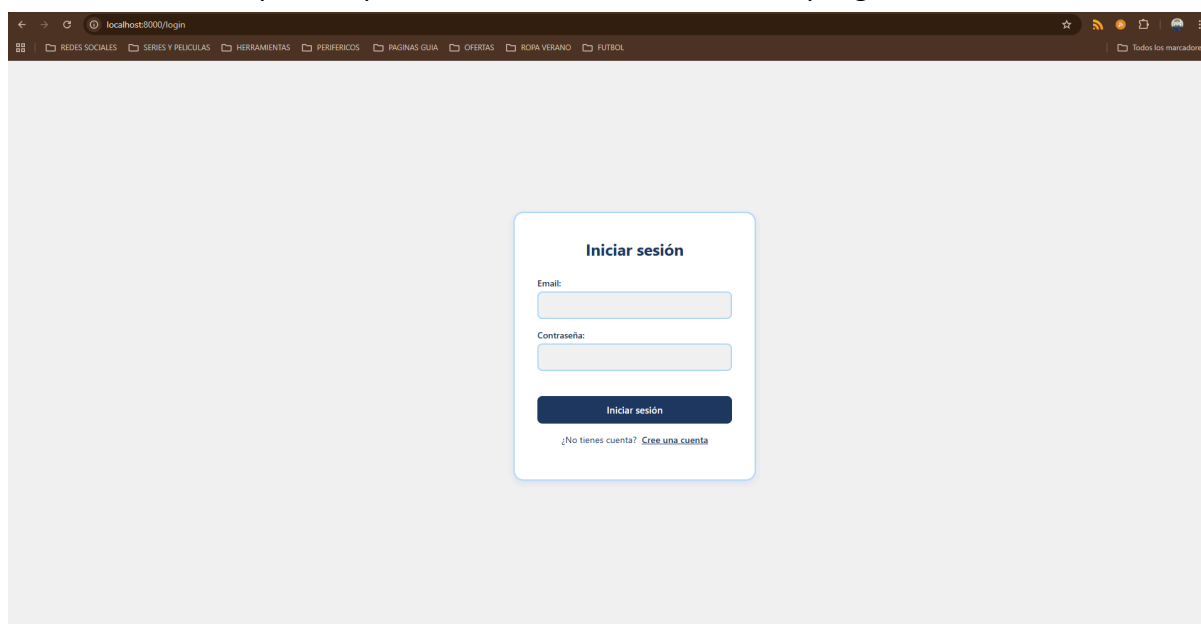
Este comando crea todas las tablas en database/migrations/.

- Paso 5. Iniciar servidor de compilación de los componentes React y frontend.

```
docker compose exec app npm run dev --host
```

Este comando inicia Vite (empaquetado) en modo desarrollo.

- Paso 6. Comprobar que funcione correctamente el despliegue local.



DESPLIEGUE EN ENTORNO DE PRODUCCIÓN

1. ¿Qué es Render?

Render es una plataforma de hosting en la nube que ofrece despliegue automático desde repositorios de GitHub que puede soportar de manera nativa aplicaciones con contenedores Docker y que está gestionada por el motor de base de datos PostgreSQL. A parte de ofrecer certificados SSL de manera automática (HTTPS)-

2. Arquitectura en Render

El despliegue de la aplicación en Render se compone de dos servicios:

- Web Service (conectaftp-web): servidor web que hostea la aplicación mediante contenedores Docker y certificado SSL cuyo dominio es: conectaftp-web.onrender.com

- Database (conectaftp-db): base de datos PostgreSQL con conexión segura mediante variables de entorno.

KEY	VALUE
APP_DEBUG	
APP_ENV	
APP_KEY	
APP_NAME	
APP_URL	
DATABASE_URL	
DB_CONNECTION	
DB_DATABASE	
DB_HOST	

Imagen representativa de la conexión mediante variables de entorno de la base de datos en el Web Service de Render

3. Configuración de Render y el entorno de producción

- Dockerfile.prod: instrucciones de las instalaciones de las dependencias y reglas que debe cumplir en producción.

```
FROM php:8.2-fpm

# Instalar dependencias del sistema
RUN apt-get update && apt-get install -y \
    git \
    zip \
    unzip \
    curl \
    libzip-dev \
    libpng-dev \
    libonig-dev \
    libxml2-dev \
    libpq-dev \
    libicu-dev \
    nginx

# Extensiones PHP necesarias para Laravel
RUN docker-php-ext-configure intl && \
    docker-php-ext-install \
        pdo_pgsql \
        pgsql \
        mbstring \
        zip \
        exif \
        pcntl \
        bcmath \
        intl

# Instalar Composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

# Instalar Node.js y npm
RUN curl -fsSL https://deb.nodesource.com/setup_20.x | bash - \
    && apt-get install -y nodejs

# Copiar archivos del proyecto
COPY ./src /var/www
WORKDIR /var/www

# Crear .env temporal con PostgreSQL para el build
RUN echo "APP_KEY=base64:${openssl rand -base64 32}" > .env && \
    echo "DB_CONNECTION=pgsql" >> .env && \
    echo "DB_HOST=localhost" >> .env

# Instalar dependencias de PHP
RUN composer install --no-dev --optimize-autoloader --no-interaction --prefer-dist
```

Imagen representativa de la configuración de Dockerfile.prod

- render.yaml: definición de la infraestructura del despliegue en Render.

```
services:
- type: web
  name: conectafp-web
  runtime: docker
  dockerfilePath: ./docker/apache/Dockerfile.prod
  dockerContext: .
  plan: free
  envVars:
    - key: APP_NAME
      value: ConectaFP
    - key: APP_ENV
      value: production
    - key: APP_DEBUG
      value: false
    - key: APP_KEY
      generateValue: true
    - key: APP_URL
      value: https://conectafp-web.onrender.com
    - key: DB_CONNECTION
      value: pgsql
    - key: DB_HOST
      fromDatabase:
        name: conectafp-db
        property: host
    - key: DB_PORT
      fromDatabase:
        name: conectafp-db
        property: port
    - key: DB_DATABASE
      fromDatabase:
        name: conectafp-db
        property: database
    - key: DB_USERNAME
      fromDatabase:
        name: conectafp-db
        property: user
    - key: DB_PASSWORD
      fromDatabase:
        name: conectafp-db
        property: password
  databases:
    - name: conectafp-db
      databaseName: conecta_fp
      user: conectafp_user
      plan: free
```

Imagen representativa del fichero render.yaml

- start.sh: script en bash para ejecutar el contenedor en Render. En este archivo se define lo siguiente:
 - Creación de directorios y permisos.
 - Generación de .env de manera dinámica.
 - Limpieza de caché.
 - Tiempo de espera hasta que inicia la base de datos (PostgreSQL).
 - Población de datos esenciales como los grados formativos, sectores empresariales, centros educativos y departamentos de los centros educativos.
 - Inicio de los servicios: PHP y Nginx.

```
#!/bin/bash

cd /var/www

# Asegurar que somos root al inicio para cambiar permisos
if [ "${id -u}" = "0" ]; then
    # Crear directorios
    mkdir -p storage/framework/sessions storage/framework/views storage/framework/cache storage/logs bootstrap/cache

    # Establecer permisos como root
    chown -R www-data:www-data storage bootstrap/cache
    chmod -R 777 storage
    chmod -R 775 bootstrap/cache

    # Limpiar archivos viejos
    rm -rf bootstrap/cache/*.php 2>/dev/null || true
    rm -rf storage/framework/cache/* 2>/dev/null || true
    rm -rf storage/framework/views/* 2>/dev/null || true
    rm -rf storage/logs/*.log 2>/dev/null || true
fi

# Crear .env con las variables de entorno de Render
cat > .env << EOF
APP_NAME="${APP_NAME}"
APP_ENV="${APP_ENV}"
APP_KEY="${APP_KEY}"
APP_DEBUG="${APP_DEBUG}"
APP_URL="${APP_URL}"

DATABASE_URL="${DATABASE_URL}"

SESSION_DRIVER=file
CACHE_STORE=file
QUEUE_CONNECTION=sync
LOG_CHANNEL=stack
LOG_LEVEL=error
EOF

# Limpiar cachés con artisan
php artisan config:clear || echo "Config clear failed, continuing..."
php artisan route:clear || echo "Route clear failed, continuing..."
php artisan view:clear || echo "View clear failed, continuing..."
php artisan cache:clear || echo "Cache clear failed, continuing..."

# Función para esperar a PostgreSQL
wait_for_db() {
    echo "Esperando a que la base de datos esté disponible..."
    max_attempts=60
}
```

Imagen representativa del fichero start.sh

4. Despliegue en Render

Se ha desplegado de la siguiente manera:

- Paso 1. Dar clic en el Web Service (conectaftp-web).

SERVICE NAME	STATUS	RUNTIME	REGION	UPDATED
conectaftp-db	✓ Available	PostgreSQL 18	Oregon	2min
conectaftp-web	✓ Deployed	Docker	Oregon	1h

- Paso 2. Dar clic en “Manual deploy” y elegir “deploy latest commit” y esperar a que se inicie el contenedor.

WEB SERVICE

conectaftp-web Docker Free Upgrade your instance → Blueprint managed

Service ID: srv-d/us78p2pic73c78ahg

OliviaDev / conectaftp main

<https://conectaftp-web.onrender.com>

Connect Manual Deploy

Deploy latest commit

Deploy a specific commit

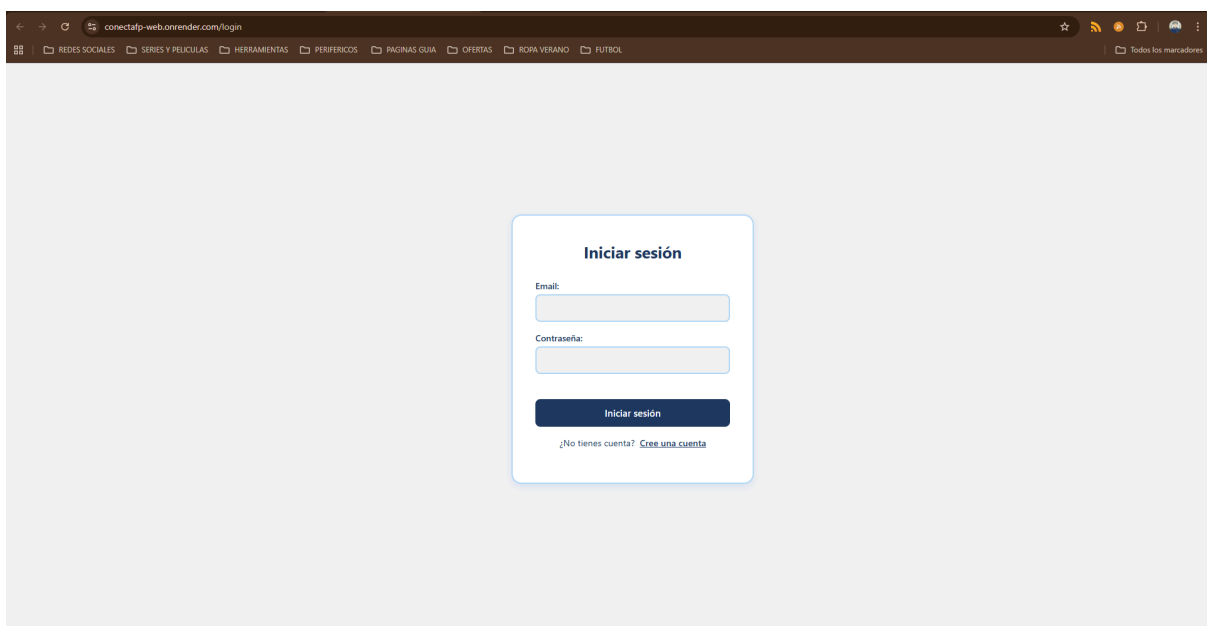
Clear build cache & deploy

Restart service

ⓘ Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

```
01:13:21 PM      ==> Your service is live 🎉
01:13:21 PM      ==>
01:13:21 PM      ==> //////////////////////////////////////
01:13:21 PM      ==>
01:13:21 PM      ==> Available at your primary URL https://conectafp-web.onrender.com
01:13:21 PM      ==>
01:13:21 PM      ==> //////////////////////////////////////
```

- Paso 3. Comprobar que funcione correctamente el despliegue mediante el dominio: <https://conectafp-web.onrender.com>.



11. ANEXO - MANUAL DE USUARIO

ConectaFP es una aplicación web diseñada para conectar alumnos de Formación Profesional en España con empresas que ofrecen prácticas. La plataforma tiene tres tipos de usuario: alumnos, profesores y empresas.

Primeros pasos

1. Elige que tipo de usuario eres (alumno, profesor o empresa).

¿Cómo vas a usar ConectaFP?


Alumno


Profesor


Empresa

2. Una vez elegido, regístrate completando el formulario.

Crear cuenta

Nombre de la empresa:

Contraseña:

Email:

Teléfono:

Sector:

-- Selecciona un sector --

NIF:

Descripción de la empresa:

Dirección:

Web:

[Atrás](#) [Crear cuenta](#)

¿Ya tienes cuenta? [Inicie sesión](#)

¿Ya tienes cuenta? [Inicie sesión](#)

Cuenta creada correctamente.

3. Una vez registrado, damos click a “Inicie sesión” (abajo del formulario) e iniciamos sesión con nuestra cuenta y redirecciona a la feed de la aplicación (proximamente).

¿Ya tienes cuenta? [Inicie sesión](#)

Iniciar sesión

Email:

Contraseña:

Iniciar sesión

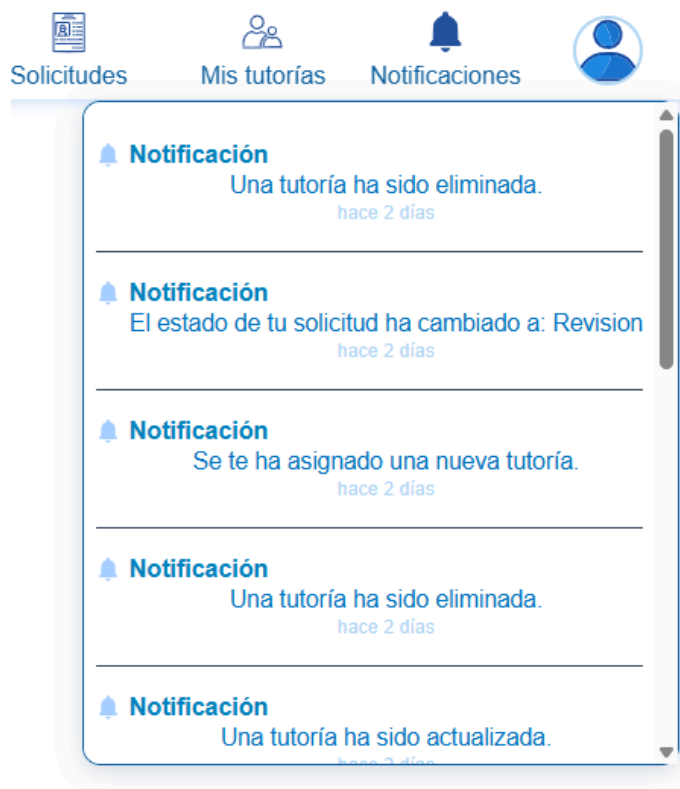
¿No tienes cuenta? [Cree una cuenta](#)



En construcción...

Funcionalidades de todos los usuarios

- Notificaciones: al hacer clic en “Notificaiones” en el encabezado, el usuario podrá ver sus notificaciones de cada acción que realiza la empresa, el alumno o el profesor asignados.



- Cerrar sesión: al hacer clic en el avatar del usuario, el usuario podrá cerrar sesión.

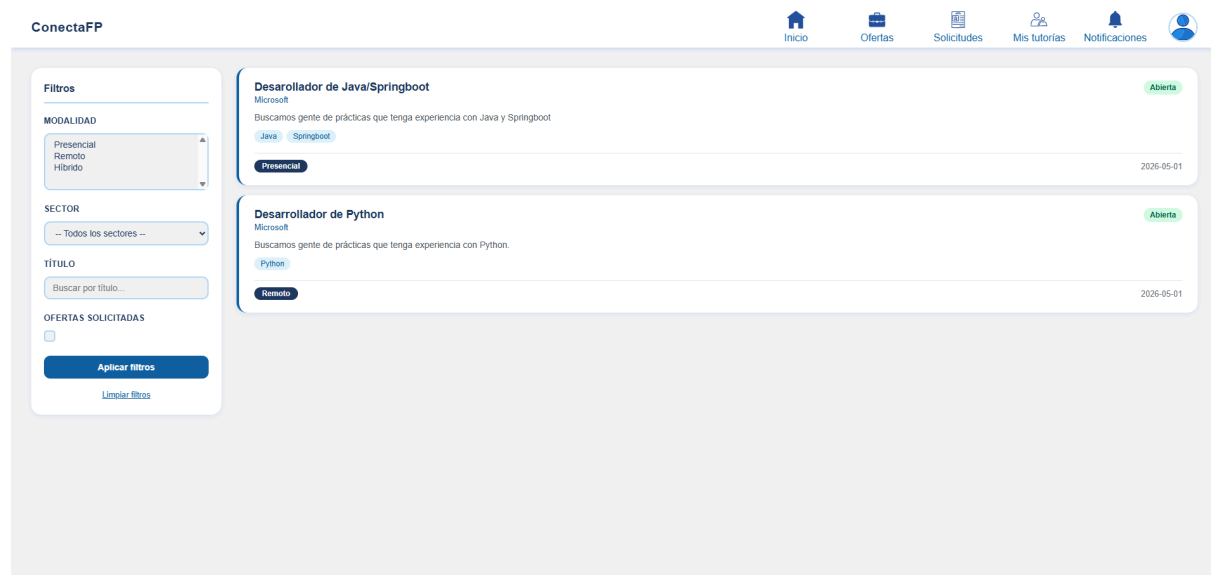


Funcionalidades del alumno

Si el usuario ha iniciado sesión siendo un alumno, puede realizar las siguientes acciones:

Ofertas

- Ver ofertas disponibles de todas las empresas.



- Filtrar ofertas según la modalidad, sector, título y/o ofertas ya solicitadas por el profesor asignado.

Filtros

MODALIDAD

Presencial
Remoto
Híbrido

SECTOR

-- Todos los sectores --

TÍTULO

OFERTAS SOLICITADAS

☐

Aplicar filtros

[Limpiar filtros](#)

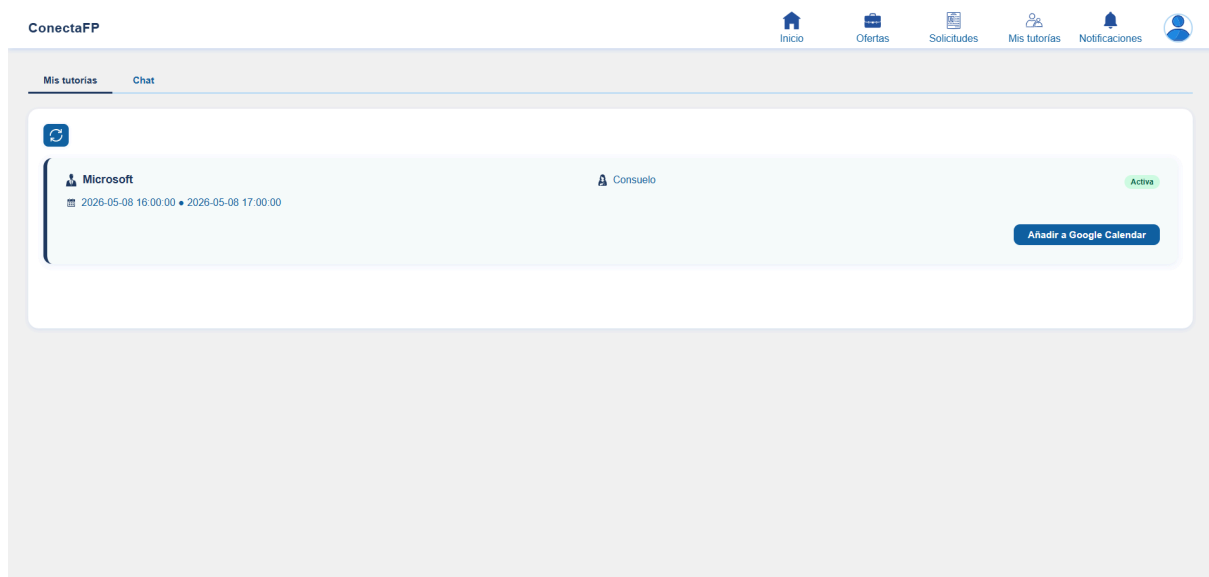
Solicitudes

- Ver las solicitudes que han sido solicitadas por el profesor asignado.

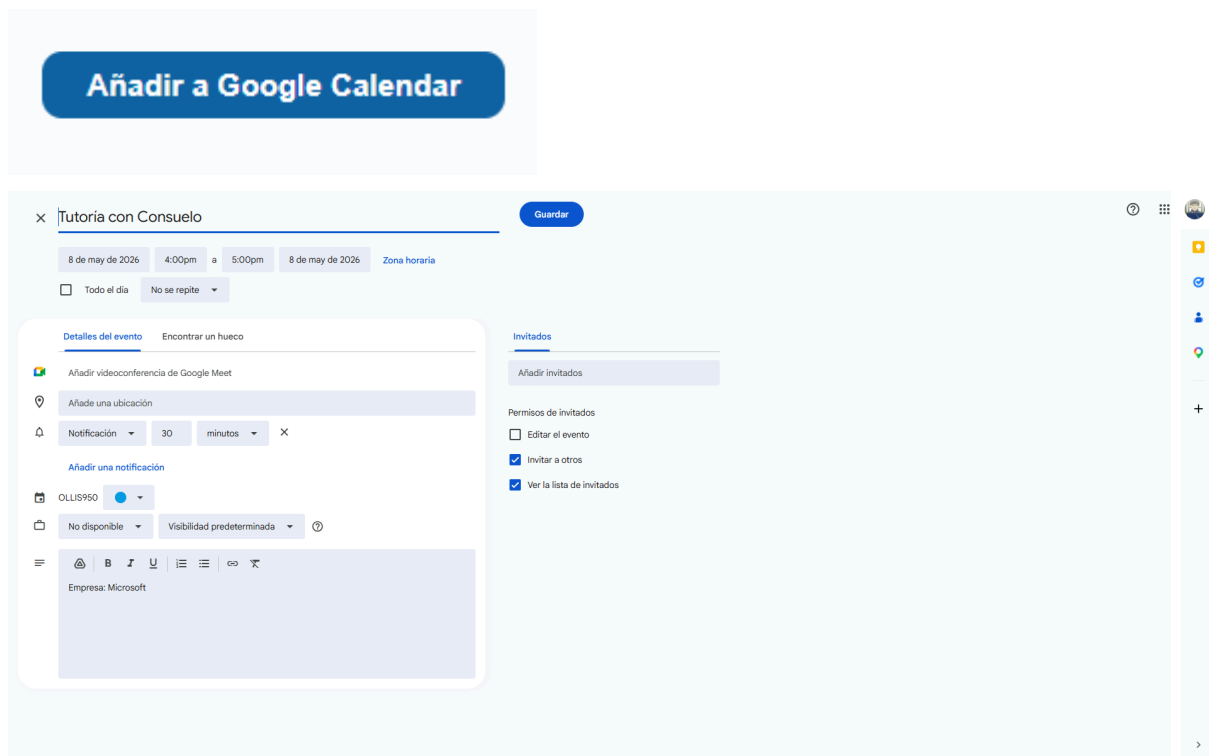
ConectaFP		 Inicio	 Ofertas	 Solicitudes	 Mis tutorías	 Notificaciones	
Solicitudes							
TÍTULO	EMPRESA	PROFESOR	FECHA	ESTADO			
Desarrollador de Java/Springboot	Microsoft	Consuelo	2026-05-01 10:08:37	Revision			
Desarrollador de Python	Microsoft	Consuelo	2026-05-01 10:52:48	Pendiente			

Mis tutorías

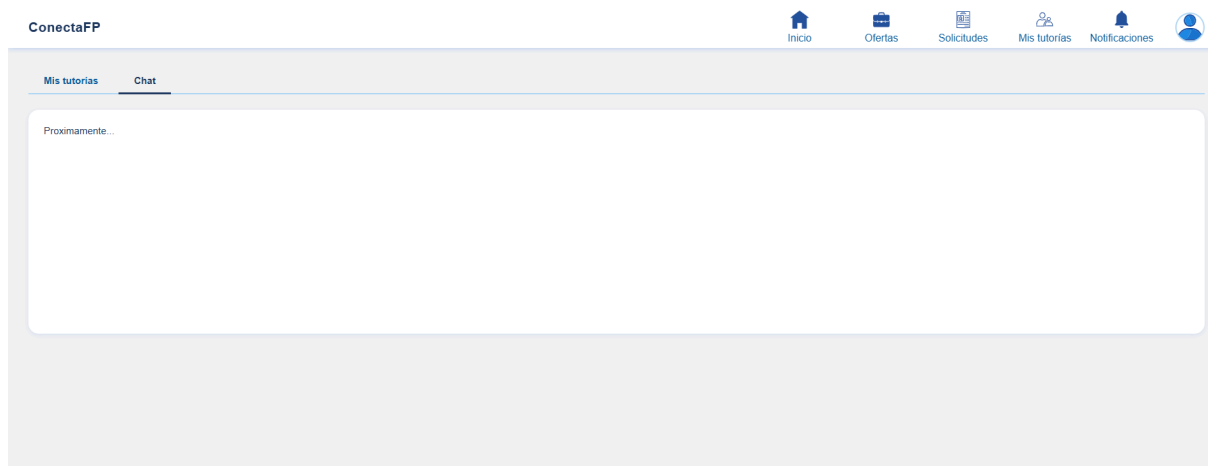
- Ver las tutorías asignadas con el profesor.



- Añadir al calendario de Google Calendar la tutoría.



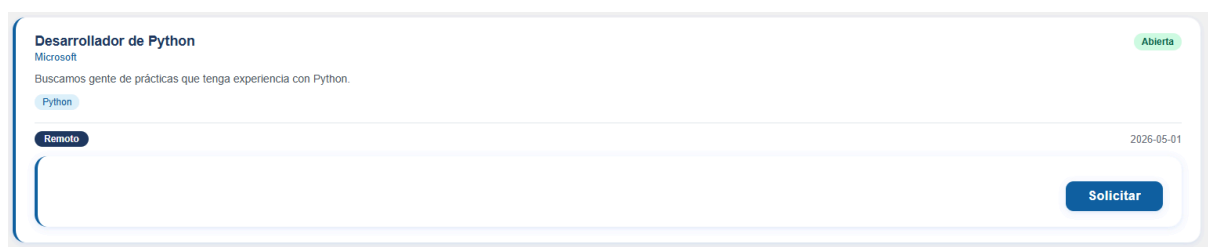
- Chat para comunicarse con el profesor sobre las prácticas (próximamente).



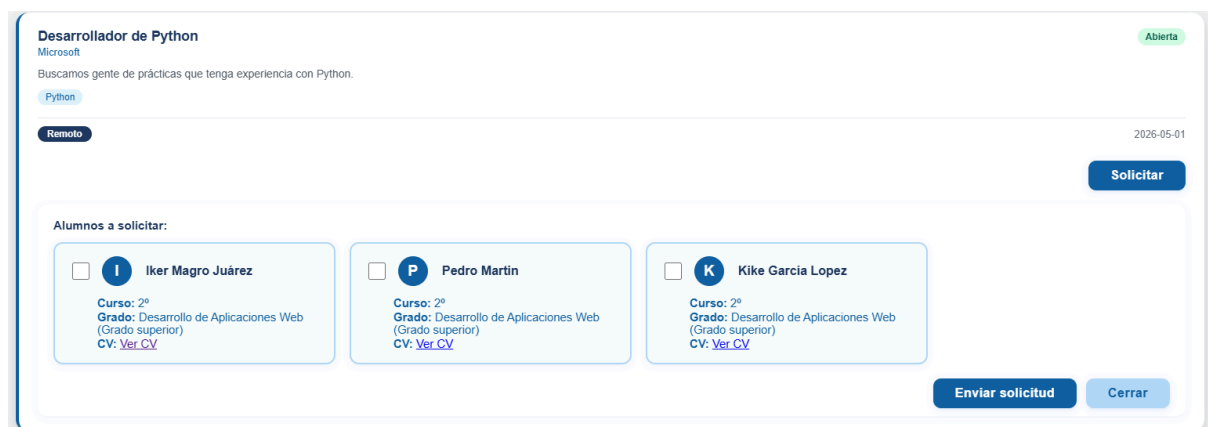
Funcionalidades del profesor

Ofertas

- Solicitar ofertas de las empresas:
1. Dar clic al botón de “Solicitar”



2. Se abre una sección donde se debe elegir uno o varios alumnos asignados para solicitar la oferta y enviarla a la empresa.



3. Una vez elegidos los alumnos a solicitar, damos clic en “Enviar solicitud”.

Alumnos a solicitar:

☐ **I** Iker Magro Juárez
 Curso: 2º
 Grado: Desarrollo de Aplicaciones Web (Grado superior)
 CV: [Ver CV](#)

☐ **P** Pedro Martin
 Curso: 2º
 Grado: Desarrollo de Aplicaciones Web (Grado superior)
 CV: [Ver CV](#)

☐ **K** Kike Garcia Lopez
 Curso: 2º
 Grado: Desarrollo de Aplicaciones Web (Grado superior)
 CV: [Ver CV](#)

Solicitud enviada correctamente.

[Enviar solicitud](#) [Cerrar](#)

Solicitudes

- Ver las solicitudes enviadas a la empresa.

ConectaFP

Inicio Ofertas Mis tutorías Solicitudes Notificaciones

Solicitudes

TÍTULO	EMPRESA	ALUMNO	FECHA	ESTADO
Desarrollador de Java/Springboot	Microsoft		2026-05-01 10:06:37	Aceptada
Desarrollador de Python	Microsoft		2026-05-01 10:52:48	Pendiente

Tutorías

- Ver las tutorías con los alumnos

Mis tutorías Crear tutoría Chat

2026-05-08 16:00:00 • 2026-05-08 17:00:00

Iker
 Activa

- Crear tutorías:
- 1. Rellenar el formulario. Solo podremos asignar a alumnos que tengan aceptada la solicitud de una oferta de una empresa.

Alumno asignado:

Iker Magro Juárez x | v

Empresa asignada del alumno:

Microsoft x | v

Fecha de inicio de la tutoría:

07/05/2026 17:00 📅

Fecha de finalización de la tutoría:

07/05/2026 17:30 📅

Estado de la tutoría:

Activa v

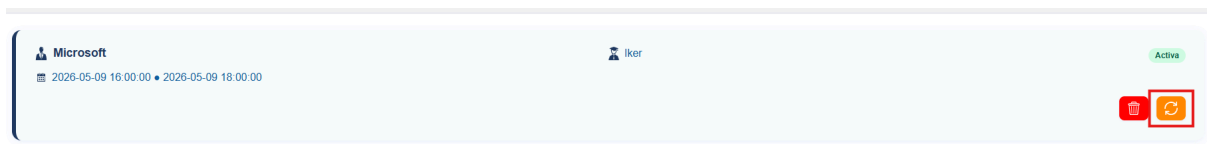
Crear tutoría

- 2. Una vez rellenado el formulario, damos clic a “Crear tutoría”.

Crear tutoría

La tutoria se ha creado correctamente.

- Editar la fecha del evento y el estado de la tutoría:



Editar tutoría

Comienzo:

08/05/2026 16:00

Fin:

08/05/2026 17:00

Estado:

Activa

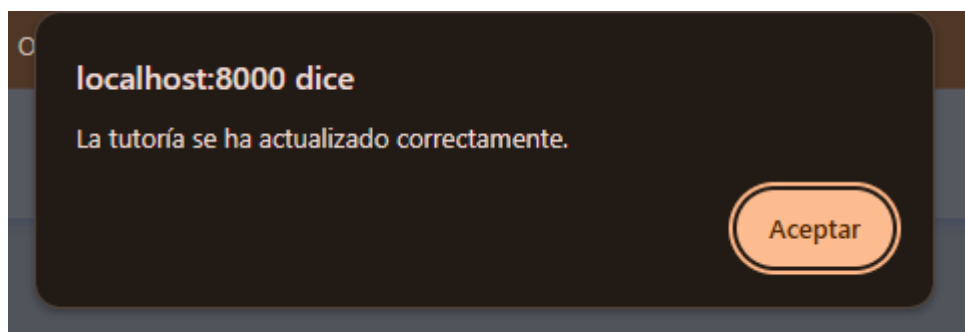
📅

📅

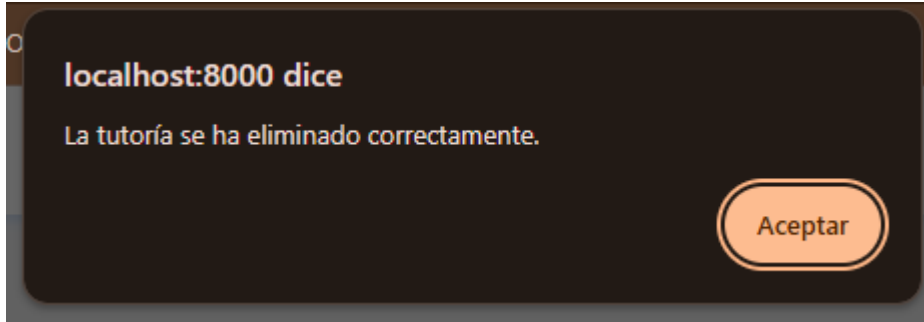
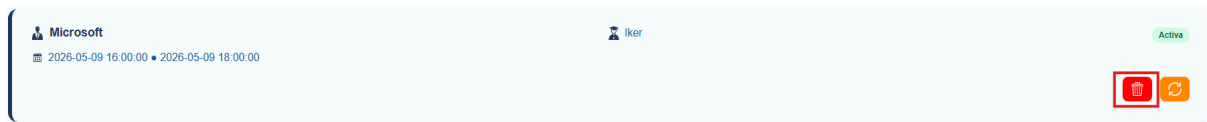
▼

Guardar

Cerrar



- Eliminar tutoría:



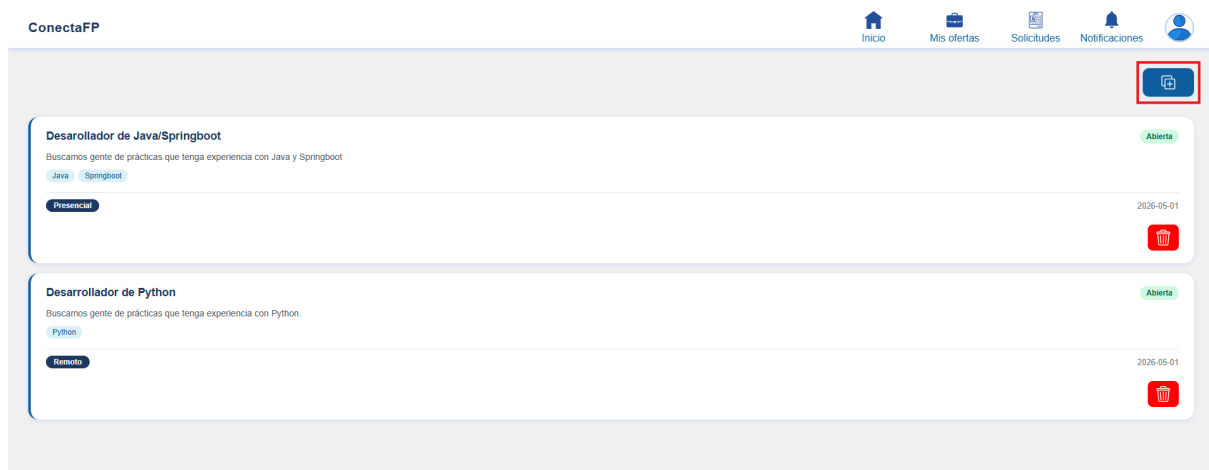
Funcionalidades de la empresa

Mis ofertas

- Ver las ofertas creadas por la empresa.



- Crear oferta:



Crear oferta

Título de la oferta:

Descripción de la oferta:

Requisitos:

Modalidad:

-- Selecciona una modalidad --

Crear oferta

Cerrar

Crear oferta

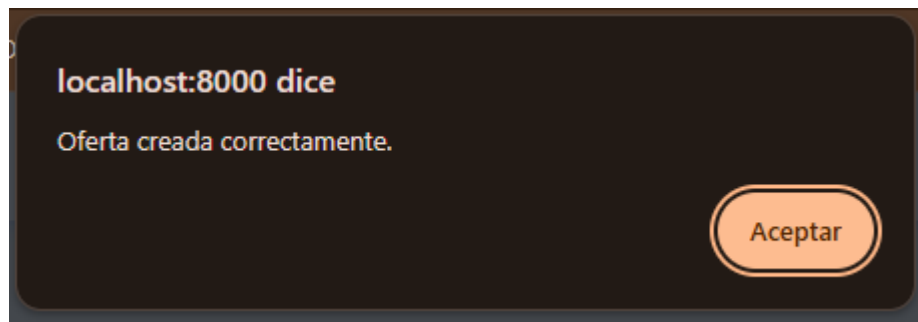
Título de la oferta:
Desarrollador de Frontend

Descripción de la oferta:
Buscamos gente de prácticas que tenga experiencia en la parte de Frontend

Requisitos:
HTML, CSS, JavaScript, React, Node.js

Modalidad:
Híbrido

Crear oferta **Cerrar**



Solicitudes

- Ver las solicitudes que han enviado los profesores sobre las ofertas de la empresa.

ConectaFP

Inicio Mis ofertas Solicitudes Notificaciones

Solicitudes

TÍTULO	ALUMNO	CV	PROFESOR	CENTRO EDUCATIVO	FECHA	ESTADO	ACCIONES
Desarrollador de Java/Springboot	Iker Magro Juárez	Ver CV	Consuelo	IES Laguna de Joazei	2026-05-01 10:08:37	Aceptada	
Desarrollador de Java/Springboot	Pedro Martin	Ver CV	Consuelo	IES Laguna de Joazei	2026-05-04 15:15:36	Aceptada	
Desarrollador de Java/Springboot	Kike García Lopez	Ver CV	Consuelo	IES Laguna de Joazei	2026-05-05 10:05:57	Pendiente	
Desarrollador de Python	Iker Magro Juárez	Ver CV	Consuelo	IES Laguna de Joazei	2026-05-01 10:52:48	Pendiente	
Desarrollador de Python	Pedro Martin	Ver CV	Consuelo	IES Laguna de Joazei	2026-05-07 15:34:19	Pendiente	
Desarrollador de Python	Kike García Lopez	Ver CV	Consuelo	IES Laguna de Joazei	2026-05-07 15:34:19	Pendiente	

- Editar el estado de la solicitud:

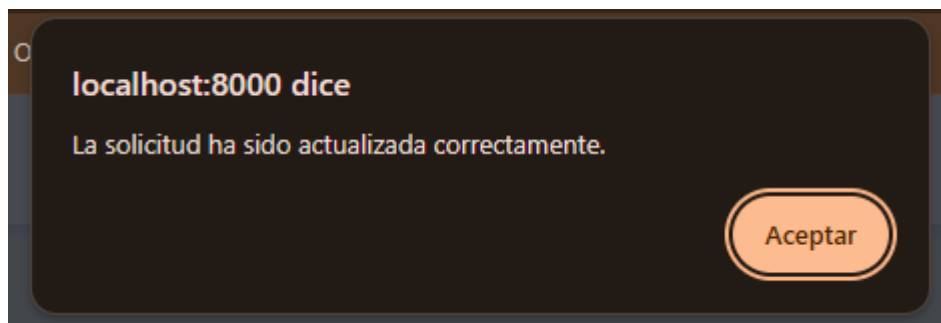
Desarrollador de Java/Springboot	Kike García Lopez	Ver CV	Consuelo	IES Laguna de Joazei	2026-05-05 10:05:57	Pendiente	
----------------------------------	-------------------	------------------------	----------	----------------------	---------------------	-----------	--

Editar solicitud

Estado:

Aceptada ▼

Actualizar solicitud Cerrar



- Ver CV del alumno asignado del profesor:

TÍTULO	ALUMNO	CV	PROFESOR
Desarrollador de Java/Springboot	Iker Magro Juárez	Ver CV	Consuelo

CURRICULUM - Iker Magro Juárez

1 / 1 100% +



Iker Magro Juárez

CONTACTO

- 661153989
- imagrojuarez@gmail.com
- Valdemoro (Madrid)
- linkedin.com/in/ikermagrojuarez
- github.com/OllisDev

INFORMACIÓN

- Ganas de aprender.
- Trabajador.
- Puntual.
- Responsable.

IDIOMAS

ESPAÑOL Nativo
INGLÉS Nivel Bajo (en proceso de mejora)

FORMACIÓN

GRADO SUPERIOR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA (2023 - 2025)

MI PERFIL

Soy estudiante del Grado Superior en Desarrollo de Aplicaciones Web (DAW) y titulado como Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM). Cuento con experiencia en lenguajes como Java, Python, PHP, C#, HTML, CSS y JavaScript.

He desarrollado proyectos como una aplicación de gestión de bibliotecas de videojuegos, que incorpora un sistema de recompensas para mejorar la experiencia del usuario.

EXPERIENCIA LABORAL

- Valle del Miro - Contrato de prácticas (marzo 2025 - mayo 2025)

LENGUAJES

- JAVA
- PYTHON
- KOTLIN
- C#
- HTML
- JAVASCRIPT
- PHP
- CSS

CONTROL DE VERSIONES

- GIT/GITHUB

ESTRUCTURA DE DATOS

- XML
- JSON
- ORACLE DB
- MYSQL

ENTORNOS DESARROLLO

